

Modeling and Implementation of a Photovoltaic cell battery charger system

by

Mehmet Gök

B.S., Electrical and Electronics Engineering, Boğaziçi University, 2023

M.S., Electrical and Electronics Engineering, Boğaziçi University, 2026

Submitted to the Institute for Graduate Studies in
Science and Engineering in partial fulfillment of
the requirements for the degree of
Master of Science

Graduate Program in Electrical and Electronics Engineering
Boğaziçi University

2026

Modeling and Implementation of a Photovoltaic cell battery charger system

APPROVED BY:

Prof. Günhan Dünder
(Thesis Supervisor)

DATE OF APPROVAL: DD.MM.YYYY

ACKNOWLEDGEMENTS

The culmination of this thesis would not have been possible without the support, guidance, and encouragement of numerous individuals who have contributed to my academic journey.

First and foremost, I would like to express my deepest gratitude to my supervisor, Prof. Günhan Dünder, for his visionary guidance, continuous encouragement, and invaluable mentorship throughout this research. I am also profoundly grateful to Dr. Kemal Ozanoğlu for his extensive help, constructive feedback, and the expertise he so graciously shared.

My sincere thanks go to the undergraduate researchers, Yasin Demir and Yasin Öztürk, whose dedication and meticulous efforts were instrumental in the successful realization of this project. Furthermore, I would like to extend my heartfelt appreciation to my laboratory mates—Uygar Yıldız, Utku Arda Akıncı, Doğa Mert Aydoğan, Mert Ceylan, and Erol Gedik—for their camaraderie, collaborative spirit, and for fostering such a stimulating research environment.

Last but certainly most importantly, my deepest gratitude belongs to my family and my beloved. Their endless patience, unconditional support, and unwavering belief in me have been the true foundation of my perseverance and success.

ABSTRACT

Modeling and Implementation of a Photovoltaic cell battery charger system

This thesis presents the modeling, design, and implementation of a hardware-efficient three-input Fuzzy Logic Controller (FLC) for Maximum Power Point Tracking (MPPT) in photovoltaic (PV) battery charging systems. The proposed architecture evaluates the change in PV voltage, change in output current, and absolute PV voltage to achieve highly responsive and accurate tracking under dynamically changing environmental conditions. A primary objective of this research is to optimize the digital control block by replacing resource-intensive arithmetic operations, such as hardware multipliers and dividers, with simple look-up tables and bitwise shift approximations. This architectural optimization significantly reduces the silicon footprint and computational complexity, making the design highly suitable for low-cost Application-Specific Integrated Circuit (ASIC) implementations and enabling the integration of multiple controllers on a single chip. The complete system, including a custom mixed-signal analog front-end and the synthesized digital controller, is validated through comprehensive Analog Mixed-Signal (AMS) simulations in the Cadence Virtuoso environment. The results demonstrate the robustness of the proposed method, achieving a steady-state PV tracking efficiency exceeding 99% and a power conversion efficiency of approximately 96%, alongside rapid transient recovery during sudden irradiance changes.

ÖZET

Fotovoltaik Hücre Batarya Şarj Sistemi Modelleme ve Uygulaması

Bu tez, fotovoltaik (PV) batarya şarj sistemlerinde Maksimum Güç Noktası Takibi (MPPT) için donanım açısından verimli, üç girişli bir Bulanık Mantık Denetleyicisinin (FLC) modellenmesini, tasarımını ve uygulanmasını sunmaktadır. Önerilen mimari, dinamik olarak değişen çevresel koşullar altında son derece duyarlı ve doğru bir takip sağlamak için PV voltajındaki değişimi, çıkış akımındaki değişimi ve mutlak PV voltajını değerlendirir. Bu araştırmanın temel amacı, donanım çarpıcıları ve bölücüleri gibi kaynak yoğun aritmetik işlemleri basit arama tabloları (LUT) ve bit düzeyinde kaydırma yaklaşımlarıyla değiştirerek dijital kontrol bloğunu optimize etmektir. Bu mimari optimizasyon, silikon alanını ve hesaplama karmaşıklığını önemli ölçüde azaltarak tasarımı düşük maliyetli Uygulamaya Özel Entegre Devre (ASIC) uygulamaları için son derece uygun hale getirir ve tek bir çip üzerinde birden fazla denetleyicinin entegrasyonuna olanak tanır. Özel bir karma sinyal analog ön uç ve sentezlenmiş dijital denetleyici içeren tüm sistem, Cadence Virtuoso ortamında kapsamlı Analog Karma Sinyal (AMS) simülasyonları ile doğrulanmıştır. Sonuçlar, önerilen yöntemin sağlamlığını göstermekte olup, ani ısınım değişiklikleri sırasında hızlı geçici toparlanmanın yanı sıra %99'u aşan sürekli hal PV takip verimliliği ve yaklaşık %96 güç dönüşüm verimliliği elde edildiğini kanıtlamaktadır.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
ÖZET	v
LIST OF FIGURES	ix
LIST OF TABLES	xi
LIST OF CODE SNIPPETS	xii
LIST OF SYMBOLS	xiii
LIST OF ACRONYMS/ABBREVIATIONS	xv
1. INTRODUCTION	1
1.1. Renewable Energy and Photovoltaic Systems	1
1.2. Maximum Power Point Tracking Algorithms	1
1.2.1. Perturb and Observe (P&O)	2
1.2.2. Incremental Conductance (IncCond)	2
1.2.3. Fuzzy Logic Control (FLC)	2
1.3. Proposed Solution	3
2. MATLAB MODEL	6
2.1. MATLAB Model Overview	6
2.2. Photovoltaic Cell Model	7
2.3. Buck Converter	8
2.4. Fuzzy Logic Controller	9
2.4.1. Sampling	10
2.4.2. Rule Firing	11
2.4.3. Duty Adjustment	15
2.5. Simulation Results	16
3. CADENCE MODEL IMPLEMENTATION	19
3.1. PV Model Implementation in Verilog-A	20
3.2. Fuzzy Logic Controller	24
3.2.1. Fuzzification	24

3.2.2.	Inference	26
3.2.3.	Defuzzification	29
3.2.4.	Duty Integration and Bumpless Transfer	31
3.3.	Simulation Results	34
3.4.	System Block Diagram	35
4.	ANALOG IMPLEMENTATION	38
4.1.	ADC	38
4.1.1.	Sample and Hold Switch	39
4.1.2.	CapDAC	40
4.1.3.	Preamplifier	45
4.1.3.1.	Simulation Results	46
4.1.4.	Comparator	49
4.1.4.1.	Offset and Noise	50
4.1.5.	SAR Logic	51
4.1.5.1.	Window Generator	52
4.1.5.2.	Comparator Clock Generator	53
4.1.5.3.	Bit Generator	54
4.1.6.	ADC Buffer	54
4.1.7.	LDO	57
4.1.8.	ADC Top-Level Performance	58
4.2.	Duty Cycle Generator	60
4.2.1.	DAC	61
4.2.2.	DAC Buffer	62
4.2.3.	Ramp Generator	63
4.2.4.	Continuous Comparator	64
4.3.	Bias and Bandgap Reference	65
5.	DIGITAL IMPLEMENTATION	66
5.1.	Hardware-Friendly Fuzzy Logic Architecture	66
5.1.1.	Difference Calculation	66
5.1.2.	Fuzzification	68
5.1.2.1.	Z-Shaped Membership Function	68

5.1.2.2.	S-Shaped Membership Function	70
5.1.2.3.	Triangular-Shaped Membership Function	72
5.1.2.4.	Fuzzifier Module	75
5.1.3.	Inference Engine	78
5.1.4.	Defuzzification	81
5.1.5.	Duty Integration	84
5.1.6.	Fuzzy Top Module	86
5.2.	Parameter Configuration and SPI Integration	91
6.	CONCLUSION	92
	REFERENCES	96

LIST OF FIGURES

1.1	Fuzzy Logic PV Charger Overview.	5
2.1	MATLAB Model Block Diagram.	6
2.2	PV Cell I-V and P-V Characteristics.	8
2.3	Fuzzy Logic Controller Block Diagram.	10
2.4	Overall MATLAB simulation results showing the three performance phases.	17
2.5	Zoomed-in views of transient events.	18
3.1	System Layout in Cadence.	19
3.2	I-V and P-V Characteristics of the Verilog-A PV Model.	23
3.3	Cadence Mixed-Signal Simulation Results showing MPP Tracking.	35
3.4	System Block Diagram.	36
4.1	Fully Differential 12-bit Asynchronous SAR ADC Architecture.	38
4.2	Bootstrapped Sample and Hold Switch Implementation.	39
4.3	DFT of the Signal Passed Through the Sample and Hold Switch.	40
4.4	CapDAC Linearity Simulation Results.	44
4.5	Fully Differential Preamplifier with CMFB Circuit.	46
4.6	CMFB Loop Stability Simulation.	47
4.7	Output Common Mode Voltage Monte Carlo Simulation.	47
4.8	Differential Open Loop AC Response.	48
4.9	Input Referred Offset Monte Carlo Simulation.	48
4.10	Input Referred Noise Simulation.	49
4.11	Comparator Input Referred Offset Monte Carlo Simulation.	50
4.12	Comparator Input Referred Noise Simulation.	51
4.13	Asynchronous SAR Logic Overview with Level Shifters.	52
4.14	Window Generator Logic Implementation.	53
4.15	Asynchronous Comparator Clock Generation Logic.	53
4.16	Timing Diagram of the Asynchronous Operation.	54
4.17	Bit Generator Logic Implementation.	54

4.18	Schematic of the normal operational amplifier used in the unity-gain stage.	55
4.19	Schematic of the Fully Differential Amplifier (FDA) stage.	56
4.20	Schematic of the overall instrumentation amplifier structure.	56
4.21	Schematic design of the Low-Dropout (LDO) regulator.	58
4.22	Stability plot of the LDO showing loop gain and phase margin.	58
4.23	4096-Point FFT of the ADC Output.	60
4.24	Schematic of the Duty Cycle Generator circuit.	61
4.25	Performance plot showing the Integral Non-Linearity (INL) and Differential Non-Linearity (DNL) of the 10-bit R-2R DAC.	62
4.26	Schematic of the unity-gain operational amplifier with a PMOS input stage used for the DAC buffer.	63
4.27	Schematic of the Ramp Generator circuit.	64
6.1	Top-level schematic of the fully integrated mixed-signal <code>IC_core</code>	93
6.2	Complete AMS testbench including the <code>IC_core</code> , Verilog-A PV model, and buck converter.	93
6.3	Final AMS simulation results demonstrating successful MPPT tracking.	94

LIST OF TABLES

2.1	Buck Converter Specifications	9
2.2	Fuzzy Logic Safety Rules	15
2.3	Fuzzy Logic Rules for Optimal Range (Updated to Match FIS) . .	16
3.1	Comparison of PV Model Parameters	23
4.1	Sample and Hold Switch Performance Metrics (Input: 0-5 V FS) .	40
4.2	Preamplifier Performance Summary	49
4.3	Comparator Performance Summary	51
4.4	ADC Performance Across Process Corners	60
4.5	DAC Buffer Amplifier Performance Summary	63
6.1	Summary of Final AMS Simulation Performance	94

LIST OF CODE SNIPPETS

1.1	System Parameters Initialization Script	3
2.1	Fuzzy Inference System Creation Script	12
3.1	Verilog-A PV Model Implementation	20
3.2	MF triangle SystemVerilog Implementation	25
3.3	Inference SystemVerilog Implementation	27
3.4	defuzzification SystemVerilog Implementation	30
3.5	Duty Integrator SystemVerilog Implementation	32
4.1	Split Capacitor Array Calculation Script	41
5.1	Difference Generator SystemVerilog Implementation	67
5.2	Z-Shaped Membership Function SystemVerilog Implementation	68
5.3	S-Shaped Membership Function SystemVerilog Implementation	70
5.4	Triangular-Shaped Membership Function SystemVerilog Implementation	72
5.5	Fuzzifier SystemVerilog Implementation	75
5.6	Inference Engine SystemVerilog Implementation	79
5.7	Defuzzifier SystemVerilog Implementation	81
5.8	Duty Integrator SystemVerilog Implementation	84
5.9	Fuzzy Top Module SystemVerilog Implementation	87

LIST OF SYMBOLS

C_i	Constant crisp output value of the i -th rule
C_{in}	Input capacitance
D	Duty cycle
dI	Change in photovoltaic current
dV	Change in photovoltaic voltage
f_{in}	Input frequency
f_s	Sampling frequency
f_{sw}	Switching frequency
I_0	Diode saturation current
I_L	Light-generated current
I_{pv}	Photovoltaic panel output current
I_{sc}	Short circuit current
L	Inductance
n	Ideality factor / Discrete time index
N	Total number of rules
P_{in}	Extracted input power
P_{mpp}	Maximum power at MPP
P_{out}	Output power
P_{pv}	Photovoltaic panel output power
R_s	Series resistance
R_{sh}	Shunt resistance
V_b	Battery voltage
V_{in}	Input voltage
V_{mpp}	Voltage at Maximum Power Point
V_{oc}	Open circuit voltage
V_{out}	Output voltage
V_{pv}	Photovoltaic panel output voltage
w_i	Firing strength of the i -th rule

Δ	Change or differential
ΔD	Change in duty cycle
η	Efficiency
μ	Degree of membership

LIST OF ACRONYMS/ABBREVIATIONS

ADC	Analog-to-Digital Converter
AMS	Analog Mixed-Signal
ASIC	Application-Specific Integrated Circuit
BGR	Bandgap Reference
CapDAC	Capacitive Digital-to-Analog Converter
CC	Constant Current
CMFB	Common-Mode Feedback
CMOS	Complementary Metal-Oxide-Semiconductor
CoG	Center of Gravity
DAC	Digital-to-Analog Converter
DFT	Discrete Fourier Transform
DNL	Differential Non-Linearity
ENOB	Effective Number of Bits
EOC	End of Conversion
FDA	Fully Differential Amplifier
FFT	Fast Fourier Transform
FIS	Fuzzy Inference System
FLC	Fuzzy Logic Controller
FS	Full Scale
FVF	Flipped Voltage Follower
GBW	Gain-Bandwidth Product
HDL	Hardware Description Language
I-V	Current-Voltage
I2C	Inter-Integrated Circuit
IC	Integrated Circuit
IncCond	Incremental Conductance
INL	Integral Non-Linearity
LDO	Low Dropout Regulator

LSB	Least Significant Bit
LUT	Look-Up Table
MIM	Metal-Insulator-Metal
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MPP	Maximum Power Point
MPPT	Maximum Power Point Tracking
MSB	Most Significant Bit
NMOS	N-Channel Metal-Oxide-Semiconductor
P&O	Perturb and Observe
P-V	Power-Voltage
PDK	Process Design Kit
PMOS	P-Channel Metal-Oxide-Semiconductor
PV	Photovoltaic
PWM	Pulse Width Modulation
RMS	Root Mean Square
S/H	Sample and Hold
SAR	Successive Approximation Register
SFDR	Spurious-Free Dynamic Range
SINAD	Signal-to-Noise and Distortion Ratio
SNR	Signal-to-Noise Ratio
SoC	System-on-Chip
SPI	Serial Peripheral Interface
THD	Total Harmonic Distortion
UGBW	Unity Gain Bandwidth
VCM	Common-Mode Voltage

1. INTRODUCTION

1.1. Renewable Energy and Photovoltaic Systems

The global demand for energy is rising rapidly, driven by population growth and industrialization. Concurrently, environmental concerns regarding fossil fuels have accelerated the transition towards renewable energy sources. Among these, solar energy stands out as a clean, abundant, and sustainable resource. Photovoltaic (PV) systems, which convert sunlight directly into electricity using semiconductor materials, have become a dominant technology in the renewable energy sector.

However, the efficiency of energy conversion in PV systems is heavily dependent on environmental conditions. The power output of a PV panel is non-linear and varies significantly with solar irradiance and ambient temperature. Unlike conventional power sources, a PV array does not have a constant internal impedance. Consequently, to extract the maximum available power, the load impedance must be dynamically matched to the source impedance. This necessity leads to the implementation of Maximum Power Point Tracking (MPPT) controllers.

1.2. Maximum Power Point Tracking Algorithms

Maximum Power Point Tracking (MPPT) is a critical technique in photovoltaic (PV) systems used to maximize power extraction under all conditions. The P-V characteristic of a solar panel is non-linear and varies with solar irradiance and temperature. There is a unique point on the P-V curve, known as the Maximum Power Point (MPP), where the PV system produces maximum power. MPPT algorithms adjust the electrical operating point of the modules to keep them at the MPP. Several algorithms have been developed to achieve this, varying in complexity, sensor requirements, convergence speed, and cost.

1.2.1. Perturb and Observe (P&O)

The Perturb and Observe (P&O) algorithm is one of the most widely used methods due to its simplicity and ease of implementation. It operates by periodically perturbing (increasing or decreasing) the terminal voltage of the PV array and comparing the PV output power with that of the previous cycle. If the power increases, the perturbation is continued in the same direction. If the power decreases, the direction is reversed. While simple, P&O suffers from oscillations around the MPP in steady state and can track in the wrong direction during rapidly changing atmospheric conditions.

1.2.2. Incremental Conductance (IncCond)

The Incremental Conductance (IncCond) method is based on the fact that the slope of the PV array power curve is zero at the MPP, positive on the left of the MPP, and negative on the right. It compares the instantaneous conductance (I/V) with the incremental conductance (dI/dV). Although IncCond offers better tracking accuracy and stability compared to P&O, particularly under rapidly changing conditions, it requires more complex control circuitry and higher computational power.

1.2.3. Fuzzy Logic Control (FLC)

Fuzzy Logic Control (FLC) has emerged as a powerful alternative for MPPT, especially in systems where the mathematical model is difficult to derive or is highly non-linear. FLC operates based on linguistic rules and does not require precise knowledge of the system parameters. It is robust against parameter variations and environmental noise. Compared to P&O and IncCond, FLC offers faster convergence and reduced oscillations around the MPP, making it highly suitable for battery charging applications where efficiency and battery health are paramount.

1.3. Proposed Solution

Based on the comparison of MPPT techniques presented in the previous section, this thesis proposes a photovoltaic battery charger system that employs a Fuzzy Logic Controller (FLC). While P&O is simple, it suffers from oscillations, and IncCond, while accurate, is computationally intensive. FLC offers a balance of robustness, fast convergence, and adaptability to the non-linear characteristics of PV panels without requiring complex mathematical modeling.

Although FLC is an excellent choice and specific architectures like Sugeno (rather than Mamdani) are highly suitable for on-chip implementation, direct realization still relies on resource-hungry logic. Operations such as complex dividers and multipliers used in the weighted average calculations consume significant silicon area. To address this, the proposed solution not only utilizes the computationally efficient Sugeno inference model but also further optimizes the digital footprint by replacing these arithmetic operations with hardware-friendly bitwise shifts and simple look-up tables.

The proposed system integrates the PV panel with a DC-DC buck converter controlled by the FLC algorithm to charge a battery efficiently. This approach ensures optimal energy harvesting even under rapidly changing atmospheric conditions.

The global system parameters, including component values for the buck converter and the control loop sampling rates, are initialized using a dedicated MATLAB script, as shown in Code Snippet 1.1. This script establishes the baseline values used across all simulations.

Code Snippet 1.1. System Parameters Initialization Script

```
%% =====
% 235W PV Buck Converter (TSMC 130nm Control)
% Topologies & Formulas Sources:
% [1] R. Erickson, D. Maksimovic, "Fundamentals of Power
```

```

    Electronics", 2nd Ed.
% [2] M. H. Rashid, "Power Electronics Handbook", 4th Ed.
% [3] Texas Instruments, "Basic Calculation of a Buck
    Converter" (SLVA477B).
% =====

clear; clc;

%% 1. System Operating Points
f_sw = 500e3;          % 500 kHz (Selected for Tape-out Logic
    density)
P_max = 235;           % 235 Watts (User Spec)
Vin_mpp = 30.6;        % PV Voltage at MPP
Vout_nom = 14.4;       % Battery Charging Voltage

% Max Output Current ( $I = P/V$ )
% Designed for worst-case low battery voltage (12V)
I_out_max = P_max / 12.0; % 19.5A

%% 2. Inductor Sizing
% Source: Erickson [1], Chapter 2, Equation 2.18
% Rule: Select Ripple ( $dI$ ) between 20-40% of max load [3].
Ripple_Ratio = 0.30;
Delta_I_L = I_out_max * Ripple_Ratio; % ~6A
D_nom = Vout_nom / Vin_mpp;           % Duty Cycle

% Formula:  $L = (V_{out} * (1 - D)) / (f_{sw} * dI)$ 
L_calc = (Vout_nom * (1 - D_nom)) / (f_sw * Delta_I_L);
% Result: ~2.6 uH

%% 3. Input Capacitor Sizing
% Source: Rashid [2], Chapter 10 "DC-DC Converters", Eq 10.32
% Also derived in TI Application Report SLTA055 for input
    ripple.

```

```

% Target Input Ripple: 1% of Vin (Essential for MPPT accuracy
).

Delta_V_in = Vin_mpp * 0.01;

% Formula:  $C_{in} = (I_{out} * D * (1-D)) / (f_{sw} * dV_{in})$ 
Cin_calc = (I_out_max * D_nom * (1 - D_nom)) / (f_sw *
    Delta_V_in);

% Result: ~32 uF (Select standard 47uF)

%% 4. Sampling Frequency
% Source: Nyquist-Shannon Theorem & Feedback Control Theory
% Rule: Sampling Freq < 1/10th of Switching Freq to avoid
aliasing noise.

sampling = 20e3;      % 20 kHz

Sim_Time = 0.6; % Seconds

```

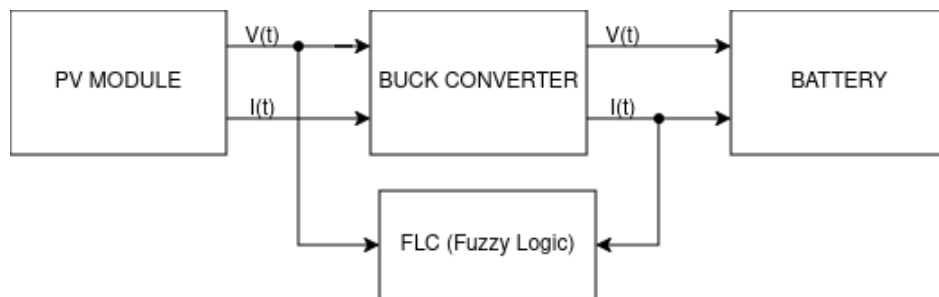


Figure 1.1. Fuzzy Logic PV Charger Overview.

2. MATLAB MODEL

2.1. MATLAB Model Overview

The design and verification of the proposed photovoltaic battery charger system were initially conducted using the MATLAB/Simulink environment. Simulink provides a comprehensive platform for Model-Based Design, allowing for the simulation of complex dynamic systems before physical implementation.

One of the primary advantages of using Simulink for this application is the availability of the Simscape Electrical library. This library offers pre-built, high-fidelity models for power electronic components, such as MOSFETs, diodes, inductors, and capacitors, as well as specialized sources like the Photovoltaic Array block. The built-in PV array model allows for precise definition of the panel's characteristics and enables the simulation of varying environmental conditions, such as changes in solar irradiance and temperature, which are crucial for testing MPPT algorithms.

Furthermore, MATLAB's Fuzzy Logic Toolbox facilitates the design and visualization of the Fuzzy Inference System (FIS). This toolbox allows for the intuitive definition of membership functions and rules, which can then be directly integrated into the Simulink simulation via the Fuzzy Logic Controller block. This integration enables rapid prototyping and tuning of the control parameters to achieve optimal tracking performance and stability.

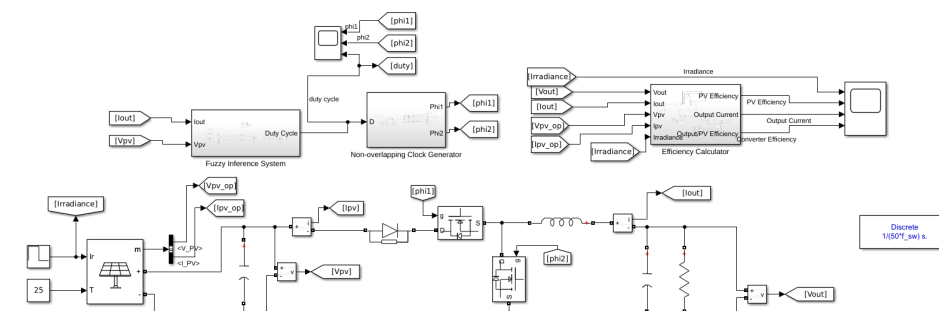


Figure 2.1. MATLAB Model Block Diagram.

2.2. Photovoltaic Cell Model

A fundamental component of the proposed system is the photovoltaic (PV) cell, which converts solar energy into electrical energy. To accurately simulate and control the charging process, a mathematical model representing the PV cell's behavior is essential. This section details the model used, specifically focusing on its non-linear current-voltage (I-V) and power-voltage (P-V) characteristics.

The output power of a PV panel is not constant; it varies significantly with environmental conditions, primarily solar irradiance and temperature. As shown in the P-V characteristics, there exists a single operating point for any given set of conditions where the power extraction is maximized. This point is referred to as the Maximum Power Point (MPP). Because the location of the MPP shifts as irradiance or temperature changes, a static connection between the panel and the load is inefficient. Therefore, it is imperative to employ Maximum Power Point Tracking (MPPT) techniques. MPPT ensures that the load impedance seen by the source is adjusted to match the optimal operating point, thereby maximizing the efficiency of energy harvesting.

Figure 2.2 illustrates the I-V and P-V characteristics of the modeled PV cell. The plot explicitly displays curves for three different irradiance levels, highlighting how the short-circuit current and the maximum power point scale with the intensity of the incident sunlight.

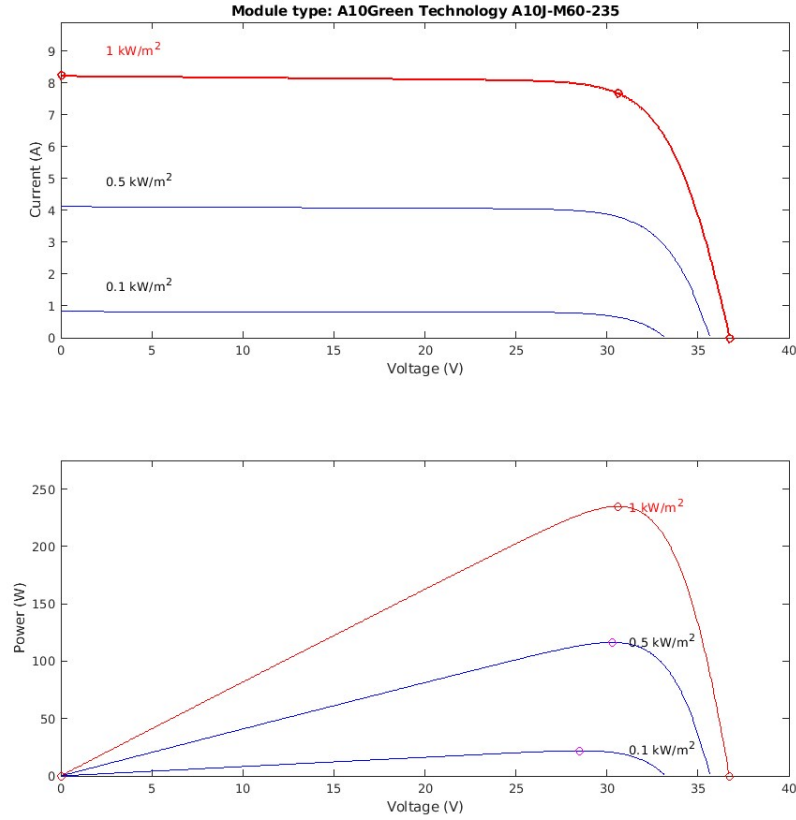


Figure 2.2. PV Cell I-V and P-V Characteristics.

2.3. Buck Converter

The buck converter is a fundamental DC-DC topology used to step down a higher input voltage to a lower output voltage while maintaining high efficiency. It operates by chopping the input voltage using a controlled switch (typically a MOSFET) and filtering the resulting signal with an inductor and a capacitor. The ratio of the output voltage to the input voltage is primarily determined by the duty cycle of the switch.

In this specific application, the photovoltaic panel operates at a Maximum Power Point (MPP) voltage of approximately 30 V. However, the system is designed to charge a 12 V battery. Consequently, a buck converter is employed to bridge this voltage difference. This configuration not only steps down the voltage but also allows the

MPPT algorithm to adjust the effective load impedance seen by the PV panel by varying the duty cycle, thereby maximizing power extraction.

The design parameters for the buck converter, as defined in the system's MATLAB model initialization script (Code Snippet 1.1), are presented in Table 2.1. These specifications are derived based on the design guidelines and component selection criteria described in [1–3].

Table 2.1. Buck Converter Specifications

Parameter	Symbol	Value
Input Voltage (MPP)	V_{in}	30.6 V
Output Voltage (Battery)	V_{out}	14.4 V
Switching Frequency	f_{sw}	500 kHz
Inductance	L	2.6 μ H
Input Capacitance	C_{in}	47 μ F

2.4. Fuzzy Logic Controller

To address the non-linear characteristics of the PV array and the need for robust tracking under changing environmental conditions, this thesis employs a Fuzzy Logic Controller (FLC). Unlike traditional linear controllers, the FLC does not require an exact mathematical model of the plant, making it highly effective for the time-varying nature of solar irradiance and temperature.

The control algorithm is based on a Sugeno-type fuzzy inference system, which is chosen for its computational efficiency and its suitability for generating a crisp control output, making it well-suited for digital implementation in control systems. The overall process of the FLC can be divided into three main stages: sampling, rule firing, and duty adjustment.

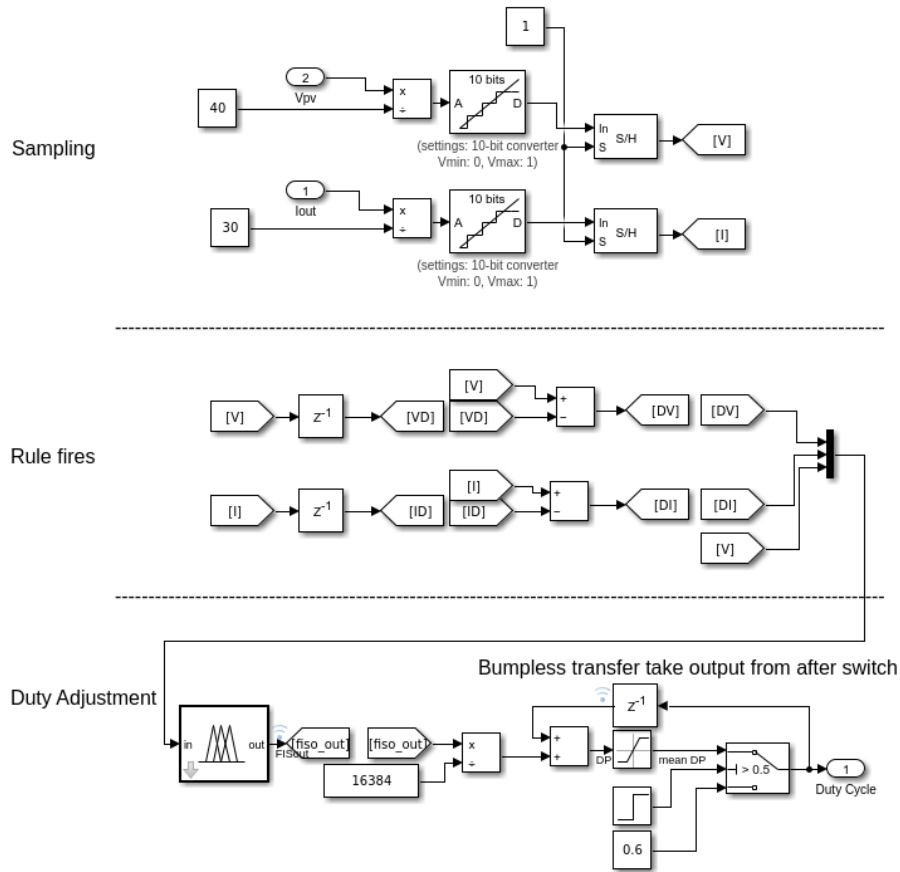


Figure 2.3. Fuzzy Logic Controller Block Diagram.

2.4.1. Sampling

The controller is implemented digitally with a sampling frequency of 20 kHz, as defined in the system parameters script (Code Snippet 1.1). At each sampling instant, the controller reads the PV voltage and current. The ADCs used for this purpose have a 10-bit resolution, which provides $2^{10} = 1024$ discrete levels (counts) to represent the analog signals. This allows for a fine-grained measurement of the PV voltage and current, which is essential for the accuracy of the MPPT algorithm. This sampling frequency is chosen to be sufficiently lower than the 500 kHz switching frequency to avoid aliasing while maintaining a fast transient response.

2.4.2. Rule Firing

The core of the FLC is the rule firing stage, where the necessary change in the duty cycle (ΔD) is determined based on the sampled inputs. This stage processes three key inputs:

- (i) **Change in PV Voltage (dV):** The difference in the PV voltage reading between consecutive samples.
- (ii) **Change in PV Current (dI):** The difference in the PV current reading between consecutive samples.
- (iii) **PV Voltage (V_{pv}):** The instantaneous PV voltage, used to determine the operating region of the PV panel.

The process involves three steps:

- **Fuzzification:** The crisp input values are converted into linguistic variables (e.g., Negative, Zero, Positive for dV and dI ; `Low_Voltage`, `Optimal_Range`, `High_Voltage` for V_{pv}) using linear and triangular membership functions.
- **Rule Evaluation:** A set of linguistic rules is evaluated to determine the control action. The rule base, summarized in Tables 2.2 and 2.3, is divided into two main groups: safety rules that make large adjustments when the voltage is too low or too high, and fine-tuning rules for operation near the MPP. A critical rule is that if both dV and dI are zero, the duty cycle is held constant to prevent oscillations at the power peak.
- **Defuzzification:** The output of the Sugeno system is a set of constant values for the change in duty cycle (ΔD), such as `Big_Decrease` (-15), `Decrease` (-1), `Hold` (0), `Increase` (+1), and `Big_Increase` (+15). The final output is a weighted average of the active rule outputs.

These rules provide a two-tiered control strategy. The safety rules (Table 2.2) trigger when the PV voltage moves into the `Low_Voltage` or `High_Voltage` regions.

These regions are defined by membership functions in the fuzzy inference system, as specified in the MATLAB script provided in Code Snippet 2.1. With a 10-bit ADC and a 40V reference, the conversion from a voltage V to an ADC count is given by: $ADC_{count} = (V/40V) \times 1024$.

Code Snippet 2.1. Fuzzy Inference System Creation Script

```
clear; clc;

% 1. Create New FIS
fis = sugfis;
fis.Name = 'MPPT_Thesis_Stable';

%% 2. Inputs: dV and dI
% dV Input (Raw ADC difference)
fis = addInput(fis, [-10 10], 'Name', 'dV_raw');
fis = addMF(fis, 'dV_raw', 'linzmf', [-5 0], 'Name', 'Negative');
fis = addMF(fis, 'dV_raw', 'trimf', [-4 0 4], 'Name', 'Zero');
fis = addMF(fis, 'dV_raw', 'linsmf', [0 5], 'Name', 'Positive');

% dI Input (Raw ADC difference)
fis = addInput(fis, [-10 10], 'Name', 'dI_raw');
fis = addMF(fis, 'dI_raw', 'linzmf', [-5 0], 'Name', 'Negative');
fis = addMF(fis, 'dI_raw', 'trimf', [-4 0 4], 'Name', 'Zero');
fis = addMF(fis, 'dI_raw', 'linsmf', [0 5], 'Name', 'Positive');

%% 3. Input: V_pv (The "Map")
% 0-1024 ADC count. Target ~783 (30.6V).
fis = addInput(fis, [0 1024], 'Name', 'V_pv');
% 1. Low Voltage: Ramp down earlier (720 to 760)
% 1. Low Voltage: Ends at 730 (Was 770)
% If V < 730, use Big Steps. If V > 730, relax.
fis = addMF(fis, 'V_pv', 'zmf', [680 730], 'Name', 'Low_Voltage');
```

```

% 2. Optimal Range: Wide Base [720 ... 850]
% This guarantees that between 730 and 840, ONLY the "Fine
    Tuning" rules (+/- 1) apply.
fis = addMF(fis,'V_pv','trimf',[720 783 850],'Name','
    Optimal_Range');

% 3. High Voltage: Starts at 840 (Was 790)
% Only panic if V > 840.
fis = addMF(fis,'V_pv','smf',[840 890],'Name','High_Voltage')
    ;

%% 4. OUTPUT: Adaptive Step Size
fis = addOutput(fis, [-20 20], 'Name', 'dDuty');

% MF Index 1: Safety Brake (-15)
fis = addMF(fis,'dDuty','constant',-15,'Name','Big_Decrease')
    ;

% MF Index 2: Fine Tune Down (-1)
fis = addMF(fis,'dDuty','constant',-1, 'Name','Decrease');

% MF Index 3: Steady State (0)
fis = addMF(fis,'dDuty','constant', 0, 'Name','Hold');

% MF Index 4: Fine Tune Up (+1)
fis = addMF(fis,'dDuty','constant', 1, 'Name','Increase');

% MF Index 5: Safety Throttle (+15)
fis = addMF(fis,'dDuty','constant', 15,'Name','Big_Increase')
    ;

%% 5. Rule Base
% [dV, dI, V_pv, Output, Weight, AND/OR]
ruleList = [];

```

```

% --- GROUP A: SAFETY (Speed) ---
ruleList = [ruleList;
    0 0 1 1 1 1]; % V=Low -> Big_Decrease (Index 1)
ruleList = [ruleList;
    0 0 3 5 1 1]; % V=High -> Big_Increase (Index 5)

% --- GROUP B: FINE TUNING (Precision) ---
ruleList = [ruleList;
    3 3 2 2 1 1; % Pos/Pos -> Decrease (-1)
    3 1 2 4 1 1; % Pos/Neg -> Increase (+1)
    1 3 2 4 1 1; % Neg/Pos -> Increase (+1)
    1 1 2 2 1 1; % Neg/Neg -> Decrease (-1)
    2 2 2 3 1 1; % If dV=0 and dI=0, do NOTHING. Stay there.
    2 1 2 3 1 1; % Stable -> Hold (0) //The "Shock Absorber
    " (The Filter Effect) ; The "Ocean" Analogy (The
    Battery Effect)
    2 3 2 3 1 1; % Stable -> Hold (0) //same effect as
    above, battery huge cap voltage can be constant and
    current can wiggle
];

fis = addRule(fis, ruleList);

%% 6. Write to File
writefis(fis, '../FIS_RULES/mppt_sugeno.fis');

```

The MPP for the modeled panel is at 30.6V (approx. ADC count 783). The safety rules are configured for large deviations from this point. For instance, the `Low.Voltage` membership function (see Code Snippet 2.1) becomes fully active for ADC counts below 730. This corresponds to a voltage of approximately 28.5V, calculated as $(730/1024) \times 40V$. If the panel voltage drops into this region, the controller applies

a **Big_Decrease** to the duty cycle, raising the panel voltage back towards the MPP. Conversely, the **High_Voltage** membership function begins to activate at an ADC count of 840, which corresponds to 32.8V $((840/1024) \times 40V)$. If the voltage exceeds this, the controller applies a **Big_Increase** to lower the panel voltage.

Once the voltage is within the **Optimal_Range**, the fine-tuning rules take over, making small adjustments to zero in on the precise MPP and hold it there, as detailed in Table 2.3.

2.4.3. Duty Adjustment

The final stage is the adjustment of the duty cycle. To ensure a smooth transition between different control states and prevent abrupt changes in the duty cycle, a bumpless transfer mechanism is implemented. This is achieved by taking the duty cycle output from the general duty cycle register ($D[n-1]$) and adding the calculated change (ΔD). This feedback loop ensures that the control signal evolves continuously from the previous state, preventing discontinuities when switching between loop dynamics.

Additionally, the integrator and the scaling of the output provide a damping effect. This lowers the impact of abrupt changes, reducing oscillations and making the system robust against noise and environmental disturbances. In the MATLAB model, the calculated ΔD is divided by a gain factor (e.g., 16384). This “lowering effect” reduces the magnitude of the duty cycle adjustments, allowing for very fine control granularity in the continuous simulation environment.

Table 2.2. Fuzzy Logic Safety Rules

PV Voltage (V_{pv})	Output (ΔD)
Low_Voltage	Big_Decrease
High_Voltage	Big_Increase

Table 2.3. Fuzzy Logic Rules for Optimal Range (Updated to Match FIS)

$dI \backslash dV$	Negative	Zero	Positive
Negative	Decrease	Hold	Increase
Zero	Hold	Hold	Hold
Positive	Increase	Hold	Decrease

2.5. Simulation Results

The performance of the proposed fuzzy logic-based MPPT controller was validated through a comprehensive simulation conducted in MATLAB/Simulink, with the overall results depicted in Figure 2.4. The total simulation time was 0.6 seconds, designed to demonstrate the controller's performance during both steady-state and transient conditions. The converter efficiency, calculated as P_{out}/P_{in} , was maintained at approximately 95% throughout the simulation. The PV efficiency, defined as P_{in}/P_{vmax} , is the primary metric for evaluating the MPPT algorithm's effectiveness.

The simulation is divided into three distinct phases, with the critical transient moments highlighted in detail in Figure 2.5.

- (i) **Initial Stabilization (0s - 0.2s):** For the first 0.2 seconds, the system operates with a fixed duty cycle. The fuzzy logic controller is inactive, allowing the converter to stabilize without active MPPT control, as seen in the first section of Figure 2.4.
- (ii) **Fuzzy Controller Activation (0.2s):** At 0.2 seconds, the fuzzy logic MPPT controller is enabled. Figure 2.5(a) shows a zoomed-in view of this event. The controller rapidly adjusts the duty cycle to find the maximum power point, achieving over 99% PV efficiency in just 13.4 milliseconds, which highlights its fast convergence speed.
- (iii) **Transient Response (0.4s):** At 0.4 seconds, a step change simulates a sudden

decrease in solar irradiance. Figure 2.5(b) details this transient. Despite the disturbance causing a momentary dip in PV efficiency to 90%, the fuzzy controller responds swiftly, restoring the efficiency to over 99% in approximately 1 millisecond.

These results confirm the robustness and high performance of the fuzzy logic controller, showcasing its ability to quickly track the MPP and maintain high efficiency, even under rapidly changing conditions.

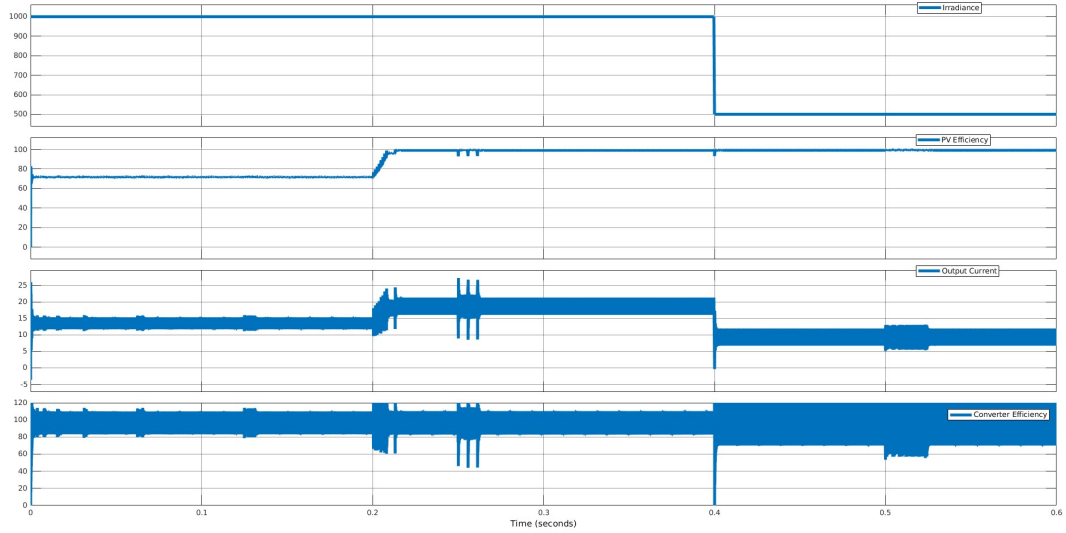
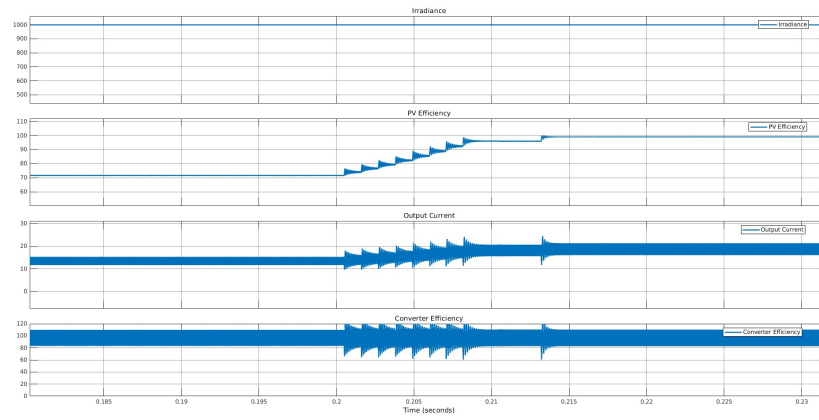
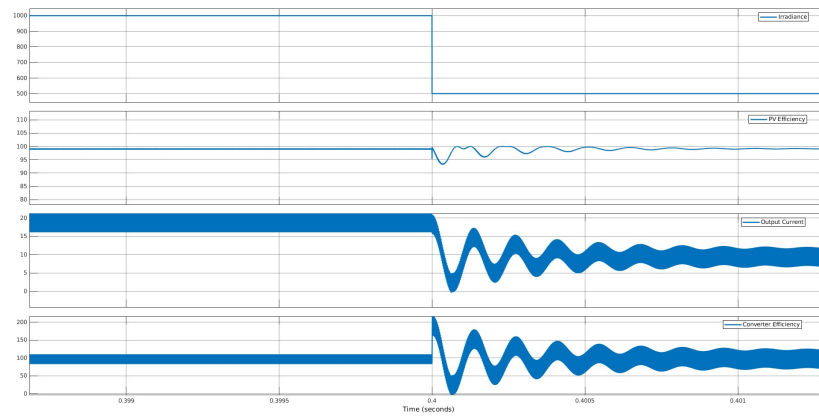


Figure 2.4. Overall MATLAB simulation results showing the three performance phases.



(a) Zoomed-in view of FLC activation at 0.2s.



(b) Zoomed-in view of transient response at 0.4s.

Figure 2.5. Zoomed-in views of transient events.

3. CADENCE MODEL IMPLEMENTATION

The initial design and verification of the fuzzy logic controller were conducted in the MATLAB/Simulink environment, as detailed in Chapter 2. While MATLAB provides a powerful platform for algorithmic development and system-level simulation, the ultimate goal is to create a physical integrated circuit. To move towards a silicon-compatible implementation, the validated model must be migrated from the abstract, high-level MATLAB environment to a hardware description language (HDL) and simulated in an environment that supports circuit-level design and verification, such as Cadence. This chapter details the process of this migration, covering the implementation of the core components in SystemVerilog and Verilog-A, and the verification of the complete system within the Cadence Virtuoso platform.

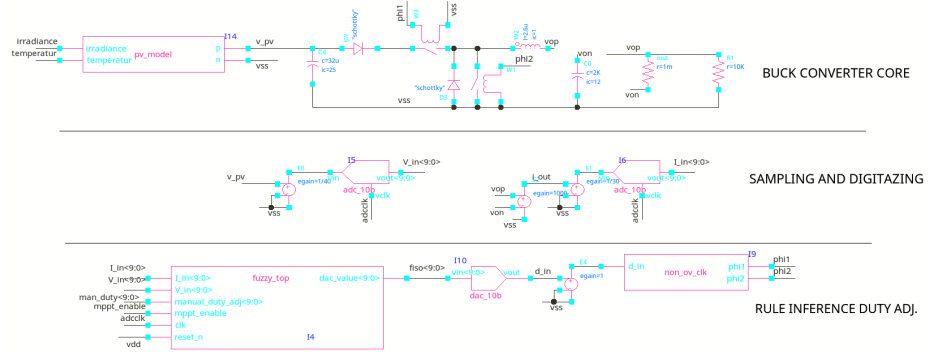


Figure 3.1. System Layout in Cadence.

The overall system layout for the Cadence implementation is shown in Figure 3.1. While some components of the system, such as the ADCs, DACs, and the power switches of the buck converter, are based on standard and well-understood designs, their detailed implementation is not the focus of this chapter. Instead, the following subsections will focus on the more complex and non-straightforward parts of the design: the implementation of the fuzzy inference engine in SystemVerilog and the behavioral modeling of the PV panel in Verilog-A.

3.1. PV Model Implementation in Verilog-A

To create a realistic simulation environment in Cadence, the photovoltaic panel was modeled using Verilog-A. This behavioral model replicates the non-linear I-V and P-V characteristics of the PV cell, allowing for accurate simulation of its interaction with the digitally-controlled buck converter. The model is based on the same mathematical equations used in the MATLAB simulation.

A widely recognized method for modeling PV panels is the five-parameter model, which uses a simplified equivalent circuit to predict the I-V curve. A significant contribution in this area was made by De Soto et al. [4], who developed and validated a method to determine the five key parameters (light-generated current I_L , diode saturation current I_0 , ideality factor n , series resistance R_s , and shunt resistance R_{sh}) from standard manufacturer datasheet values. This approach allows for accurate performance prediction across a wide range of operating conditions (irradiance and temperature), which is essential for testing MPPT algorithms.

The Verilog-A implementation, shown below, is based on this five-parameter model. It takes irradiance and temperature as inputs and dynamically calculates the panel's output current based on its terminal voltage, accurately reproducing the non-linear behavior of the PV cell.

Code Snippet 3.1. Verilog-A PV Model Implementation

```
'include "disciplines.vams"
'include "constants.vams"

// Photovoltaic Panel Model
module pv_model (irradiance, temperatur, p, n);
    inout p, n;
    input irradiance, temperatur;
    electrical p, n, irradiance, temperatur;
```

```

// --- Array Configuration ---
parameter real Ns_mod = 1;      // Series modules
parameter real Np_mod = 1;      // Parallel strings
parameter real N_cell = 60;     // Cells per module

// --- Model Parameters (Extracted from Simulink/A10Green) ---
parameter real IL_ref = 8.2383;  // Light current [A]
parameter real IO_ref = 2.8335e-10; // Saturation
current [A]
parameter real n_id = 0.98962;   // Ideality factor
parameter real Rs_ref = 0.25811;  // Series resistance
[Ohm]
parameter real Rsh_ref = 196.9278; // Shunt resistance
[Ohm]

// --- Physics & Temp Constants ---
parameter real alpha_Isc = 0.00096999; // Temperature
coefficient [A/C]
parameter real Tref = 25.0;           // Reference
temperature [C]
parameter real Eg = 1.12;             // Bandgap energy
[eV] for Si

// Internal Variables
real Tc_degC, Tc_K, Tref_K;
real G_suns, IL, IO, Vt, Rs, Rsh, V_diode;

analog begin
    // 1. Environmental Input Scaling
    G_suns = V(irradiance);
    Tc_degC = V(temperatur);
    Tc_K = Tc_degC + 273.15;

```

```

Tref_K  = Tref + 273.15;

// 2. Variable Calculation
Vt = (N_cell * Ns_mod * 'P_K * Tc_K / 'P_Q) * n_id;

// Adjust for current irradiance and temperature
IL = (IL_ref + alpha_Isc * (Tc_degC - Tref)) * G_suns
;

// Calculate saturation current temperature
dependence
I0 = I0_ref * pow((Tc_K / Tref_K), 3.0) * limexp((
    'P_Q * Eg / (n_id * 'P_K)) * (1/Tref_K - 1/Tc_K));

// Scale parasitic resistances for array size
Rs  = Rs_ref * Ns_mod / Np_mod;
Rsh = Rsh_ref * Ns_mod / Np_mod;

// 3. Kirchhoff's Current Law Implementation
// Define diode voltage including the drop across
series resistance
V_diode = V(p, n) - I(p, n) * Rs;

// Final output current: Losses (Diode + Shunt) minus
Source (Photocurrent)
I(p, n) <+ (I0 * Np_mod * (limexp(V_diode / Vt) - 1))
+
    (V_diode / Rsh) - (IL * Np_mod);

end
endmodule

```

The Verilog-A code provides a behavioral model of the PV cell. Figure [3.2](#) shows

the I-V (current-voltage) and P-V (power-voltage) characteristics produced by this model, highlighting the Maximum Power Point (MPP).

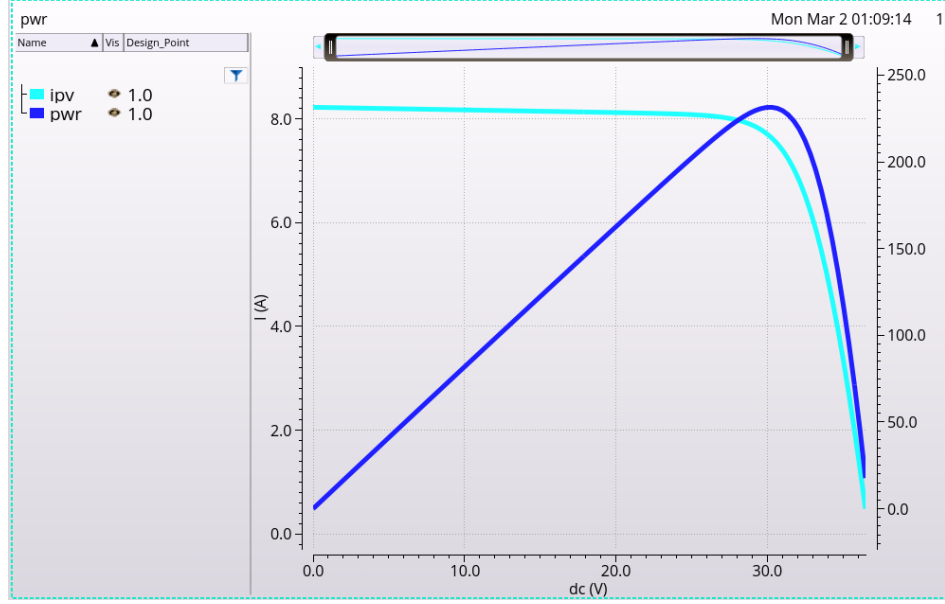


Figure 3.2. I-V and P-V Characteristics of the Verilog-A PV Model.

While the MATLAB/Simulink model serves as the ideal reference, the Verilog-A behavioral model implemented in Cadence exhibits slight deviations in the electrical characteristics. These minor discrepancies arise from the implementation of the five-parameter equations and the analog solver's interpretation. Table 3.1 compares the key parameters of the MATLAB reference model and the Verilog-A implementation.

Table 3.1. Comparison of PV Model Parameters

Parameter	MATLAB Model	Verilog-A Model
Open Circuit Voltage (V_{oc})	36.72 V	36.5 V
Short Circuit Current (I_{sc})	8.23 A	8.226 A
MPP Voltage (V_{mpp})	30.6 V	30.29 V
Maximum Power (P_{mpp})	235.008 W	231.4 W

3.2. Fuzzy Logic Controller

To prepare the design for hardware implementation, the fuzzy logic controller, previously validated in MATLAB, was implemented as a digital module using SystemVerilog. SystemVerilog is a hardware description and verification language that allows for the creation of synthesizable logic, providing a direct path to a physical integrated circuit. This implementation follows a Sugeno-type fuzzy inference model, which is well-suited for hardware due to its computational efficiency. The process is broken down into three fundamental stages: fuzzification, inference, and defuzzification.

3.2.1. Fuzzification

In the fuzzification stage, the crisp digital inputs from the ADCs, representing the PV panel's voltage (V_{pv}) and current (I_{pv}), are converted into fuzzy linguistic variables. This is achieved by evaluating their degree of membership in predefined sets (e.g., 'Low', 'Optimal', 'High').

The core of this process is the calculation of the membership degree for a given input value. For this controller, triangular membership functions are used, which are defined by three points. A triangular function is composed of two linear segments, meaning the membership degree can be calculated using the simple equation of a line. If the slope and two points of a line are known, any other point is easily calculable.

Given two points (x_1, y_1) and (x_2, y_2) , the slope (m) of the line connecting them is:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \quad (3.1)$$

With the slope and a known point, the membership degree (y) for any input (x) can

be calculated using the point-slope formula:

$$y - y_1 = m(x - x_1) \quad (3.2)$$

This linear relationship makes the calculation straightforward to implement in hardware, as shown in the SystemVerilog code below.

Code Snippet 3.2. MF triangle SystemVerilog Implementation

```
module mf_triangle #(
    parameter signed [10:0] A = -383,
    parameter signed [10:0] B = -300,
    parameter signed [10:0] C = -217
)(
    input  logic signed [10:0] x,      // input variable (ADC
        diff)
    output logic [7:0] mu              // membership degree
        (0-255)
);

    // Local parameter for scaling
    localparam logic signed [10:0] SCALE = 11'd255;
    // Internal signals
    logic signed [11:0] num1, num2, den1, den2;
    logic signed [21:0] tmp;
    logic signed [15:0] result;
    always_comb begin
        // Default outputs
        mu = 8'd0;
        // Left slope: A < x < B
        if ((x > A) && (x < B)) begin
            num1 = x - A;
            den1 = B - A;
            tmp = num1 * SCALE;
            // Rounded division: (tmp + den1/2) / den1
```

```

        result = (tmp + (den1 >> 1)) / den1;
        mu = (result > 255) ? 8'd255 : result[7:0];
    end
    // Right slope: B <= x < C
    else if ((x >= B) && (x < C)) begin
        num2 = C - x;
        den2 = C - B;
        tmp = num2 * SCALE;
        result = (tmp + (den2 >> 1)) / den2;
        mu = (result > 255) ? 8'd255 : result[7:0];
    end
    // Peak: x == B (technically covered above, but
    explicit)
    else if (x == B) begin
        mu = 8'd255;
    end
end

endmodule

```

Other common membership functions, such as the linear S-shaped and Z-shaped functions, are similarly simple to implement in SystemVerilog as they are also based on linear equations.

3.2.2. Inference

The inference engine serves as the decision-making core of the controller. It evaluates the fuzzy inputs generated in the fuzzification stage against the linguistic rules defined in the system's rule base. For this implementation, the logical 'AND' operator connecting the antecedents of a rule is realized using the minimum function, a standard method in fuzzy logic control.

Mathematically, for a rule with multiple conditions, such as:

$$\text{IF } x \text{ is } A \text{ AND } y \text{ is } B \text{ THEN } z \text{ is } C \quad (3.3)$$

The firing strength w of the rule is calculated as:

$$w = \min(\mu_A(x), \mu_B(y)) \quad (3.4)$$

In the SystemVerilog implementation, this logic is synthesized using combinational circuits. A dedicated function, `min3`, is created to compute the minimum of three values, facilitating rules that depend on dV , dI , and the V_{pv} operating range simultaneously. The code below illustrates the parallel evaluation of the safety rules and the fine-tuning rules. The safety rules (w_1, w_2) are triggered solely by the PV voltage level, while the fine-tuning rules (w_3 to w_8) are active only when the voltage is within the optimal range, ensuring precise tracking around the MPP. This implementation strictly adheres to the logic defined in Tables 2.2 and 2.3.

Code Snippet 3.3. Inference SystemVerilog Implementation

```
module inference (
    // dV and dI inputs (3 each)
    input  logic [7:0] dV_mu_neg, dV_mu_ze, dV_mu_pos,
    input  logic [7:0] dI_mu_neg, dI_mu_ze, dI_mu_pos,
    // V_pv inputs (3)
    input  logic [7:0] Vpv_mu_low, Vpv_mu_opt, Vpv_mu_high,

    // Rule Firing Strengths (w1 through w8)
    output logic [7:0] w [1:8]
);

// Internal function for 3-input MIN
function automatic logic [7:0] min3(logic [7:0] a, logic
```

```

[7:0] b, logic [7:0] c);
    logic [7:0] m2;
    m2 = (a < b) ? a : b;
    return (m2 < c) ? m2 : c;
endfunction

always_comb begin
    // --- GROUP A: SAFETY RULES ---
    // Rule 1: V=Low -> Big_Decrease (w1)
    w[1] = Vpv_mu_low;

    // Rule 2: V=High -> Big_Increase (w2)
    w[2] = Vpv_mu_high;

    // --- GROUP B: FINE TUNING (Only active in Optimal
    // Range) ---
    // Rule 3: Pos/Pos/Opt -> Decrease
    w[3] = min3(dV_mu_pos, dI_mu_pos, Vpv_mu_opt);

    // Rule 4: Pos/Neg/Opt -> Increase
    w[4] = min3(dV_mu_pos, dI_mu_neg, Vpv_mu_opt);

    // Rule 5: Neg/Pos/Opt -> Increase
    w[5] = min3(dV_mu_neg, dI_mu_pos, Vpv_mu_opt);

    // Rule 6: Neg/Neg/Opt -> Decrease
    w[6] = min3(dV_mu_neg, dI_mu_neg, Vpv_mu_opt);

    // Rule 7: Stuck (Ze/Ze/Opt) -> HOLD
    w[7] = min3(dV_mu_ze, dI_mu_ze, Vpv_mu_opt);

    // Rule 8: Stable (Ze/Any/Opt) -> HOLD (Combining
    // your two stable rules)

```

```

// We can simplify the "Stable" logic here or list
// them separately.
w[8] = min3(dV_mu_ze, (dI_mu_neg > dI_mu_pos ?
               dI_mu_neg : dI_mu_pos), Vpv_mu_opt);
end
endmodule

```

As can be seen in the `always_comb` block, the logic directly maps to the rule tables. The safety rules (`w[1]` and `w[2]`) depend only on the PV voltage, while the fine-tuning rules (`w[3]` through `w[8]`) are active only in the optimal voltage range (`Vpv_mu_opt`) and correctly implement the logic described in Table 2.3.

3.2.3. Defuzzification

The final stage of the fuzzy logic controller is defuzzification, which converts the fuzzy conclusions from the inference engine into a crisp, numerical control signal. Since this system utilizes a Sugeno-type inference model, the output membership functions are singletons (constant values) rather than distributed fuzzy sets. This simplifies the defuzzification process to a weighted average calculation, which is computationally efficient for hardware implementation.

The crisp output, representing the required change in duty cycle (ΔD), is calculated using the Center of Gravity (CoG) method for singletons:

$$\Delta D = \frac{\sum_{i=1}^N w_i \cdot C_i}{\sum_{i=1}^N w_i} \quad (3.5)$$

where w_i is the firing strength of the i -th rule, C_i is the constant output value associated with that rule, and N is the total number of rules.

The SystemVerilog implementation, shown below, defines the singleton constants corresponding to the linguistic variables `Big_Decrease` (-15), `Decrease` (-1), `Hold` (0),

Increase (+1), and **Big_Increase (+15)**. The module accumulates the weighted sum (numerator) and the total weight (denominator) to compute the final integer division.

Code Snippet 3.4. defuzzification SystemVerilog Implementation

```

module defuzzifier (
    input  logic [7:0] w [1:8],          // 8 Rule weights
        from Inference
    output logic signed [10:0] dDuty      // Final step size
);

    // Sugeno Constants from MATLAB
    // Index matches our Rule List:
    // w1:-15, w2:15, w3:-1, w4:1, w5:1, w6:-1, w7:0, w8:0
    localparam signed [7:0] C1 = -15; // Big Decrease
    localparam signed [7:0] C2 =  15; // Big Increase
    localparam signed [7:0] C3 =  -1; // Decrease
    localparam signed [7:0] C4 =   1; // Increase
    localparam signed [7:0] C5 =   1; // Increase
    localparam signed [7:0] C6 =  -1; // Decrease
    localparam signed [7:0] C7 =   0; // Hold
    localparam signed [7:0] C8 =   0; // Hold

    // Intermediate accumulators
    logic signed [20:0] sum_wi_ci;
    logic [15:0] sum_wi;

    always_comb begin
        // Weighted Sum:  $\sum(w_i * C_i)$ 
        sum_wi_ci = ( $signed({1'b0, w[1]}) * C1 ) +
                    ( $signed({1'b0, w[2]}) * C2 ) +
                    ( $signed({1'b0, w[3]}) * C3 ) +
                    ( $signed({1'b0, w[4]}) * C4 ) +
                    ( $signed({1'b0, w[5]}) * C5 ) +

```

```

        ( $signed({1'b0, w[6]}) * C6 ) +
        ( $signed({1'b0, w[7]}) * C7 ) +
        ( $signed({1'b0, w[8]}) * C8 );

// Total Weight:  $\Sigma(w_i)$ 
sum_wi = w[1] + w[2] + w[3] + w[4] + w[5] + w[6] + w
[7] + w[8];

// Weighted Average (The "Defuzz")
if (sum_wi != 0) begin
    dDuty = sum_wi_ci / $signed({1'b0, sum_wi});
end else begin
    dDuty = 0;
end
end
endmodule

```

3.2.4. Duty Integration and Bumpless Transfer

The final stage of the digital control loop is the Duty Integrator. While the fuzzy inference engine determines the required *change* in the operating point (ΔD), the Pulse Width Modulation (PWM) generator requires an absolute duty cycle value to drive the power MOSFETs. This module performs the discrete-time integration of the fuzzy output, mathematically represented as:

$$D[n] = D[n - 1] + \Delta D[n] \quad (3.6)$$

where $D[n]$ is the current duty cycle and $\Delta D[n]$ is the adjustment calculated by the defuzzifier.

This integration strategy serves two critical purposes:

- (i) **Bumpless Transfer:** We take the duty cycle output from the general duty cycle register ($D[n-1]$) as the baseline. By adding the calculated change (ΔD) to this existing global output, the system ensures a smooth transition. This prevents discontinuities that would otherwise occur when switching between different control dynamics (e.g., Safety vs. Fine-Tuning).
- (ii) **Robustness and Damping:** The integrator and damping provide the opportunity to lower oscillations and make the system robust. In the MATLAB model, this was achieved by dividing the output by a large gain factor. In the SystemVerilog implementation, where integer arithmetic prevents such division (as it would result in zero), this damping is achieved by accumulating small integer steps ($\pm 1, \pm 15$) into the high-resolution duty cycle register. This effectively lowers the impact of individual control actions, filtering out transient noise and maintaining stability around the MPP.

The SystemVerilog implementation, provided below, includes saturation logic to constrain the duty cycle within safe hardware limits. It takes the signed 8-bit output from the defuzzifier and updates the 10-bit duty cycle register on every clock cycle.

Code Snippet 3.5. Duty Integrator SystemVerilog Implementation

```

module duty_integrator (
    input  logic      clk,
    input  logic      reset_n,
    input  logic      mppt_enable,
    input  logic signed [10:0] duty_adjust,
    input  logic [9:0] manual_duty_in,
    output logic [9:0] dac_value
);

    // --- CONFIGURATION ---
    // CHANGE THIS: 6 for Turbo Sim, 14 for Real Hardware
    localparam int FRAC_BITS = 6;

```



```

// Safety Headroom: 10 (Data) + FRAC_BITS + 2 (Sign/
  Overflow)
// We keep 26 bits because it is safe for both 9 and 14.
logic signed [25:0] acc;

// --- SCALED LIMITS (Calculated automatically) ---
localparam signed [25:0] MIN_ACC = 26'd307 << FRAC_BITS;
localparam signed [25:0] MAX_ACC = 26'd716 << FRAC_BITS;
localparam signed [25:0] START_VAL = 26'd512 << FRAC_BITS
    ;

always_ff @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        acc <= START_VAL;
    end else begin

        if (mppt_enable == 1'b0) begin
            // === MANUAL MODE ===
            // FIX: Use shift operator '<<' instead of
            //      hardcoded zeros.
            // This automatically shifts by 9 or 14
            //      depending on the param.
            // We cast to 26-bit signed to ensure the
            //      width matches 'acc'.
            acc <= signed'({1'b0, manual_duty_in}) <<
                FRAC_BITS;
        end
        else begin
            // === MPPT MODE ===
            logic signed [26:0] next_acc;
            next_acc = acc + duty_adjust;

            if (next_acc > MAX_ACC)

```

```

        acc <= MAX_ACC;
    else if (next_acc < MIN_ACC)
        acc <= MIN_ACC;
    else
        acc <= next_acc[25:0];
    end
end
end

// === OUTPUT ===
// Example (FRAC=9): We want bits [18:9]
// Example (FRAC=14): We want bits [23:14]
assign dac_value = acc[FRAC_BITS + 9 : FRAC_BITS];

endmodule

```

It is important to note that the SystemVerilog code snippets provided in this section primarily serve to demonstrate the functional behavior and the algorithmic logic of the controller within the Cadence simulation environment. Direct implementation of arithmetic operations such as division—used here for the center-of-gravity defuzzification—can consume excessive logic resources and timing budget during synthesis. In a practical hardware implementation, these operations are often optimized or approximated using shifts, look-up tables, or specific architectural constraints to minimize area and power. Further details on the optimized digital implementation and synthesis considerations are discussed in Chapter 5.

3.3. Simulation Results

The functional verification of the SystemVerilog implementation was performed within the Cadence Virtuoso environment. The simulation was run for 180 ms, with the MPPT enabled at 0.3 ms.

As discussed in the duty integrator section, the selection of the damping fraction is critical. A fraction of 6 was selected; a higher value (e.g., 15) would result in excessive settling time for transient ringing. To further stabilize the inputs, V_{pv} , I_{pv} , and I_{out} are processed through a moving average block with a window width of 200 μs .

Figure 3.3 illustrates the system's response. The extracted input power (P_{in}) settles at 227 W, which is close to the theoretical maximum of 231.4 W listed in Table 3.1. The average PV voltage ($V_{pv.avg}$) stabilizes at 30.48 V, compared to the theoretical MPP voltage of 30.29 V. The output power (P_{out}) is measured at 217 W. This results in a PV efficiency (P_{in}/P_{vmax}) of $227/231.4 \approx 98.1\%$ and a converter efficiency (P_{out}/P_{in}) of $217/227 \approx 95.6\%$. Despite these minor deviations, the model demonstrates robust operation and successful tracking.

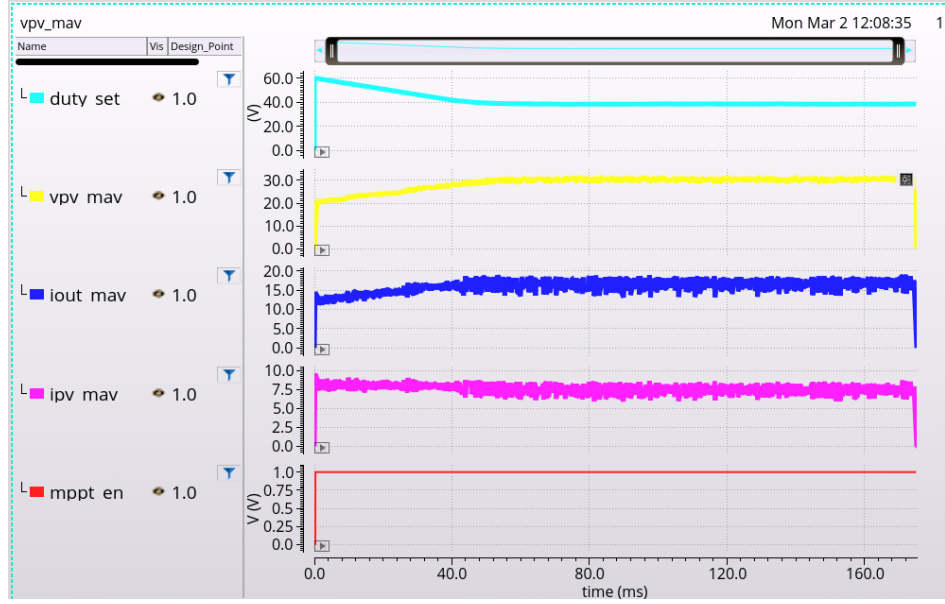


Figure 3.3. Cadence Mixed-Signal Simulation Results showing MPP Tracking.

3.4. System Block Diagram

The complete system architecture integrates the digital control logic with the analog power and sensing components. Figure 3.4 illustrates the detailed block diagram, showing the signal flow from the PV panel sensors through the mixed-signal interface to the Fuzzy Logic Controller, and the resulting control signal driving the

Buck Converter.

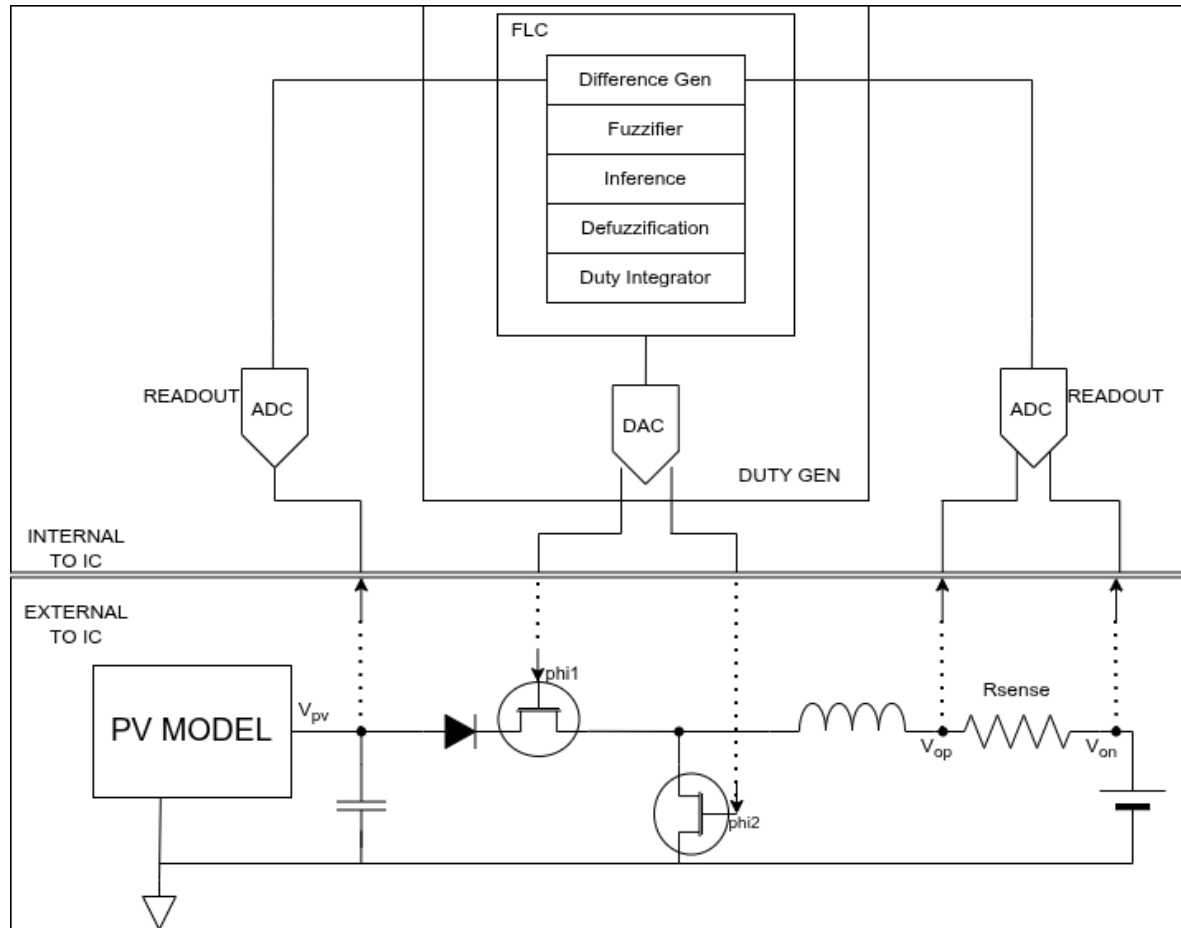


Figure 3.4. System Block Diagram.

To ensure precise tracking and stable operation, the system is decomposed into the following hierarchical requirements:

- **Digital Controller**
 - **Fuzzy Logic Controller:** Implements the Sugeno inference engine, rule base, and duty integrator.
- **Analog / Mixed-Signal Interface**
 - **Analog-to-Digital Converters (ADC)**
 - * *Resolution:* 10-bit
 - * *Sampling Rate:* 20 kSPS
 - * *Function:* Digitizes V_{pv} and I_{pv} for the controller.

- **Digital-to-Analog Converter (DAC)**
 - * *Resolution:* 10-bit
 - * *Function:* Converts the digital output of the FLC to an analog signal to drive the duty controller.
- **Analog Support & References** (Required for ADC/DAC precision)
 - **Low Dropout Regulator (LDO):** Provides a clean, regulated supply voltage to isolate the sensitive analog blocks from switching noise.
 - **Bandgap Reference (BGR):** Generates a stable, temperature-independent voltage reference required by the LDO and ADCs.
- **Power Stage & Drivers**
 - **Duty Generator (PWM Adjuster):** This block compares the analog control voltage from the DAC against a high-frequency triangular waveform to generate the PWM signal. The resulting duty cycle is proportional to the control voltage, directly modulating the power flow through the buck converter.
 - **High-Side Switch Driver:** A specialized circuit required to drive the gate of the high-side N-MOSFET. Since the MOSFET's source is not ground-referenced but connected to the switching node, this driver must provide a gate voltage that is higher than the input voltage (V_{in}) to turn the switch on. This is typically achieved using a bootstrap circuit.
 - **Low-Side Switch Driver:** This circuit drives the gate of the low-side synchronous N-MOSFET. As its source is connected to ground, the driver provides a simpler 0V to V_{DD} logic-level signal, but it must be capable of sourcing and sinking high peak currents to charge and discharge the gate capacitance quickly, minimizing switching losses.

4. ANALOG IMPLEMENTATION

4.1. ADC

The analog-to-digital converter (ADC) is a critical component in the mixed-signal feedback loop, responsible for digitizing the PV voltage and current for the digital controller. For this system, a fully differential 12-bit Successive Approximation Register (SAR) ADC was chosen due to its excellent power efficiency and suitability for medium-speed, high-resolution applications. The fully differential architecture offers superior noise immunity, rejecting common-mode noise which is prevalent in switching power converter environments.

The ADC operates asynchronously, meaning the internal bit-cycling is triggered by the completion of the previous bit decision rather than a high-speed external clock. This simplifies the clocking requirements of the system.

Figure 4.1 illustrates the top-level architecture of the implemented SAR ADC.

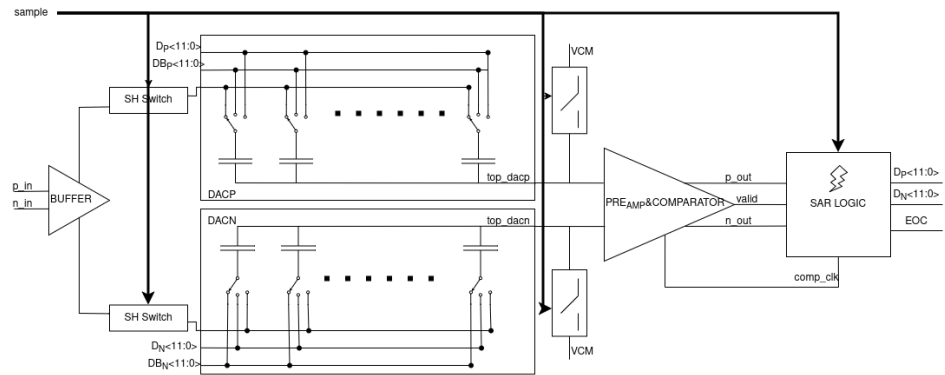


Figure 4.1. Fully Differential 12-bit Asynchronous SAR ADC Architecture.

To achieve the required linearity and speed, the ADC is composed of several critical sub-blocks, which are analyzed in detail in the following sections:

4.1.1. Sample and Hold Switch

The Sample and Hold (S/H) switch is the front-end of the ADC, responsible for acquiring the analog input signal onto the CapDAC array. To achieve high linearity over a wide input common-mode range, a bootstrapped switch topology is employed. The design is based on the well-known bootstrapped switch architecture proposed by Razavi [5], which ensures a constant gate-to-source voltage (V_{gs}) for the main sampling transistor. This minimizes signal-dependent charge injection and maintains a low, constant on-resistance, which is critical for achieving 12-bit linearity.

To further enhance robustness, the bulk connections of the transistors are critical. The final implementation of the switch is shown in Figure 4.2. The design of such high-performance switches is a well-studied area in CMOS analog design [5, 6].

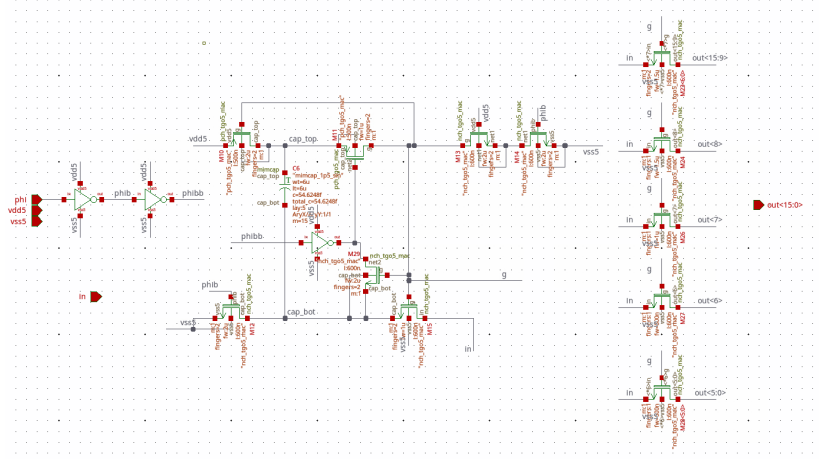


Figure 4.2. Bootstrapped Sample and Hold Switch Implementation.

The parasitic capacitance on the switching node was calculated as 60 fF in the typical process corner. Consequently, a 500 fF bootstrap capacitor was selected. Although this results in a gate-source voltage (V_{gs}) drop of approximately 10% due to charge sharing, the resulting on-resistance of the MOSFET is still sufficiently low to charge the ~ 25 pF sampling capacitor within the 20 kHz sampling period.

Linearity is the main metric for this device, as any distortion introduced at the

sampling stage directly limits the overall accuracy of the ADC. Figure 4.3 shows the 4096-point Discrete Fourier Transform (DFT) of the signal passed through the switch. The spectrum demonstrates the high linearity of the design, with harmonic components significantly suppressed relative to the fundamental tone.

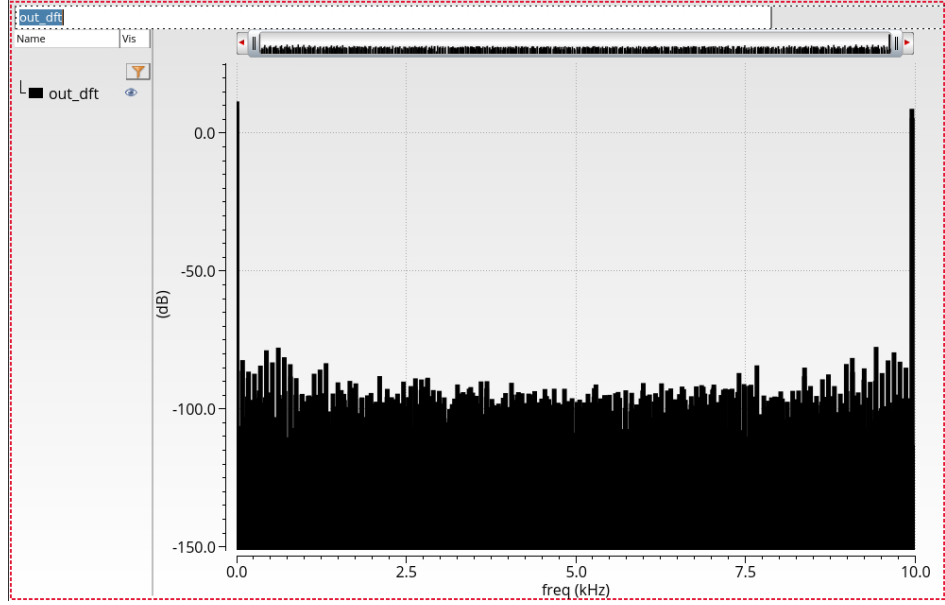


Figure 4.3. DFT of the Signal Passed Through the Sample and Hold Switch.

Table 4.1. Sample and Hold Switch Performance Metrics (Input: 0-5 V FS)

Metric	Value
THDB3	-89.12 dB
SFDR	82.2 dB

4.1.2. CapDAC

The Capacitor-DAC (CapDAC) utilizes a split-capacitor architecture to reduce the total capacitance and silicon area. Instead of a conventional 2^{12} unit capacitor array, the design employs a 6-bit MSB and 6-bit LSB split. Although the minimum Metal-Insulator-Metal (MIM) capacitor allowed by the PDK is 6 fF, a unit capacitor of 60 fF is used in this architecture. Following the equivalent value method described in [7], a bridge capacitor of 2C (twice the unit capacitance) was chosen, reducing the total number of unit capacitors to 128. To ensure proper binary weighting, an additional

dummy capacitor is required. The calculation for this dummy capacitor was performed using the MATLAB script in Code Snippet 4.1, which implements the method from [7], and resulted in a value of $62C$. This configuration significantly improves the settling time and bandwidth of the ADC.

Code Snippet 4.1. Split Capacitor Array Calculation Script

```
clear; clc;

% =====
% STEP 1: DIRECT CONFIGURATION
% =====

N = 12;      % Total Bits
L = 6;      % LSB Bits
M = 6;      % MSB Bits
m = 2;      % Bridge Capacitor Ratio ( $C_{bri} = m * C$ )
C_unit = 1; % Normalized unit cap

% =====
% STEP 2: EWM CALCULATIONS (Eq. 4 logic)
% =====

% Formula for  $C_c$ :  $(2^L - 1) * C_{bri} = 2^L * C + C_c$ 
C_bri = m * C_unit;
C_lsb_array_sum = 2^L * C_unit; % e.g.,  $64C$  for 6-bit LSB
C_c = (2^L - 1) * C_bri - C_lsb_array_sum;

fprintf('--- Configuration Results ---\n');
fprintf('Resolution: %d-bit (%d LSB + %d MSB)\n', N, L, M);
fprintf('Bridge Cap ( $C_{bri}$ ): %.1f * C\n', m);

if C_c < 0
    fprintf('WARNING: Calculated  $C_c$  is %.1f * C.\n', C_c);
    fprintf('A negative capacitor is physically impossible!\n
    ');
```

```

        fprintf('Reason: C_bri=1 is too small for a 6+6 split.
        Try m >= 1.016 or m = 2.\n\n');
    else
        fprintf('Adjustment Cap (Cc): %.1f * C\n\n', C_c);
    end

% =====
% STEP 3: WEIGHT VERIFICATION
% =====
% Generate LSB binary weights: [1, 2, 4, 8, 16, 32...]
lsb_bits = 2.^(0:L-1);
msb_bits = 2.^(0:M-1);

% Calculate Attenuation Factor (alpha)
% alpha = Cbri / (Cbri + C_lsb_total + Cc)
C_lsb_total = C_lsb_array_sum + C_c;
alpha = C_bri / (C_bri + C_lsb_total);

% Final Weights
w_lsb = alpha * lsb_bits;
w_msb = msb_bits;
all_weights = [w_lsb, w_msb];

% Linearity Check: The gap between the LSBs and MSBs
% Ideally, MSB[0] should be exactly 2x the LSB[max]
gap = w_msb(1) / w_lsb(end);
fprintf('--- Linearity Verification ---\n');
fprintf('Weight Ratio (MSB_min / LSB_max): %.4f\n', gap);
if abs(gap - 2) < 1e-6
    fprintf('RESULT: Binary Weighting is PERFECT.\n');
else
    fprintf('RESULT: Binary Weighting is BROKEN.\n');
end

```

```

% =====
% STEP 4: PLOT TRANSFER FUNCTION
% =====

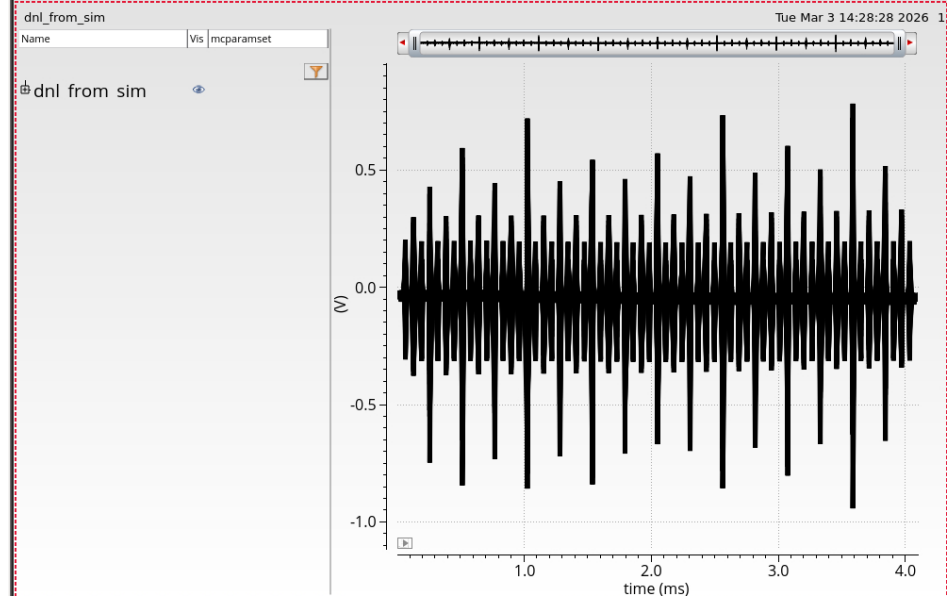
num_steps = 1000;
vin = linspace(0, sum(all_weights), num_steps);
digital_out = zeros(size(vin));

for i = 1:num_steps
    v_rem = vin(i);
    code = 0;
    for b = N:-1:1
        if v_rem >= all_weights(b)
            v_rem = v_rem - all_weights(b);
            code = code + 2^(b-1);
        end
    end
    digital_out(i) = code;
end

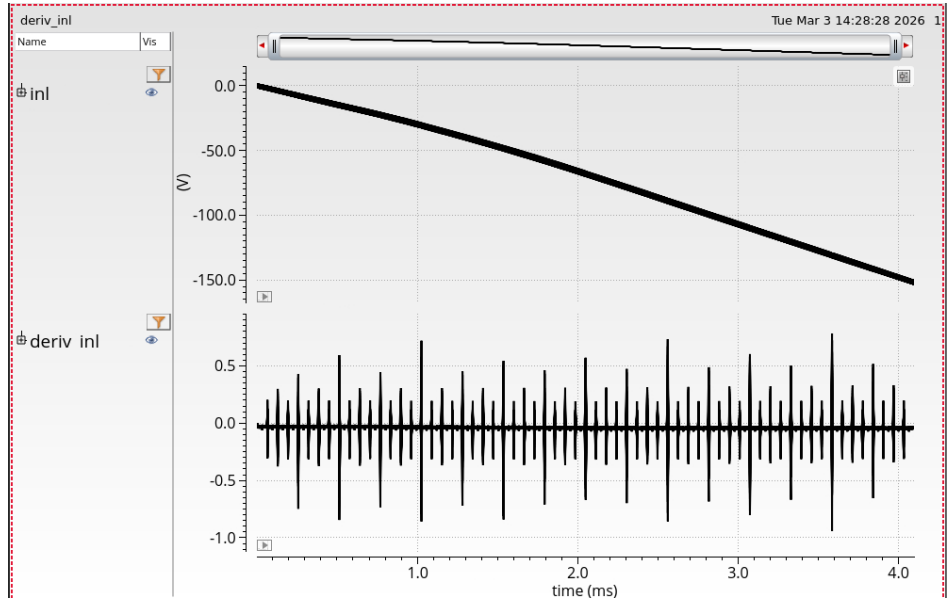
figure;
plot(vin, digital_out, 'LineWidth', 1.5);
title(sprintf('%d-bit ADC (%d+%d Split, m=%.1f)', N, L, M, m)
);
xlabel('Analog Input'); ylabel('Digital Output Code');
grid on;

```

The linearity of the CapDAC is paramount for the overall ADC performance. The two key metrics are Differential Non-Linearity (DNL), which measures the deviation in step width from the ideal 1 LSB, and Integral Non-Linearity (INL), which is the cumulative sum of DNL errors. Figure 4.4 shows the simulated DNL and INL of the designed CapDAC.



(a) DNL



(b) INL and its derivative

Figure 4.4. CapDAC Linearity Simulation Results.

The results shown in Figure 4.4 are obtained from a Monte Carlo simulation of 200 runs using a low-discrepancy sequence. The Differential Non-Linearity (DNL) is observed to be up to ± 1 LSB, which is a natural characteristic in split-capacitor arrays due to bridge capacitor sensitivity. However, since the target Effective Number of Bits (ENOB) for this application is 10 bits, this performance is considered acceptable.

The Integral Non-Linearity (INL) is also within acceptable limits, exhibiting a nearly constant gain error. This is further illustrated by the derivative of the INL, plotted as $d(INL)/dt \times 1\mu s$ (corresponding to the LSB step time). Finally, to further improve the ADC performance after fabrication, the bridge capacitor is implemented with trim capacitors. Given that the primary unit capacitance (C_{unit}) is 60 fF, these trim capacitors consist of 6 fF unit cells, corresponding to an added calibration capacitance of $0.1 \times C_{unit}$.

4.1.3. Preamplifier

In a high-resolution SAR ADC, the dynamic comparator can inject significant kickback noise back onto the sensitive CapDAC nodes when it fires. This charge injection can corrupt the voltage stored on the capacitor array, leading to erroneous bit decisions. To mitigate this effect and improve the overall linearity, a preamplifier is placed between the CapDAC and the comparator.

The preamplifier serves two primary functions:

- (i) **Isolation:** It acts as a buffer, shielding the delicate analog nodes of the CapDAC from the comparator's kickback noise.
- (ii) **Gain:** It provides a moderate amount of voltage gain, amplifying the small differential voltage from the CapDAC. This relaxes the sensitivity requirements for the subsequent comparator, reducing its effective input-referred offset and noise.

The implemented preamplifier, shown in Figure 4.5, is a fully differential amplifier that utilizes top-plate sampling. A fully differential topology is used to maintain high common-mode noise rejection throughout the signal path. To stabilize the output common-mode voltage, which is otherwise undefined in a differential amplifier, a dedicated Common-Mode Feedback (CMFB) circuit is employed. The reference voltage for the CMFB is set to $VDD/2$, which is generated by a simple resistive divider filtered

by a capacitor to reduce noise; this $VDD/2$ reference will also be used by the top-plate biasing LDO. The CMFB circuit senses the average output voltage, compares it to this reference, and adjusts the gate voltage of the PMOS load transistors to maintain a stable DC operating point.

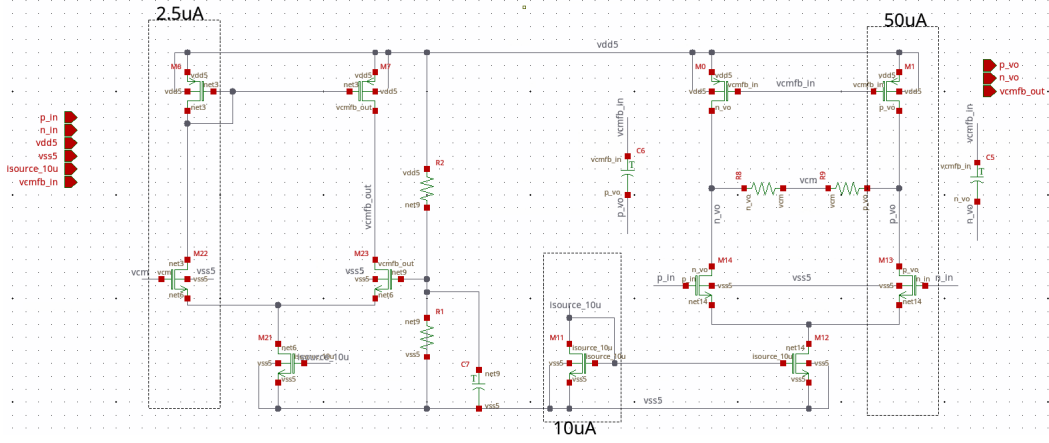


Figure 4.5. Fully Differential Preamplifier with CMFB Circuit.

4.1.3.1. Simulation Results. The performance of the preamplifier was verified through extensive simulations. The key metrics evaluated include differential and common-mode stability, input-referred offset, and noise performance.

CMFB Stability Analysis. Ensuring stability is crucial for the preamplifier. The primary concern is the stability of the Common-Mode Feedback (CMFB) loop, which must be analyzed to prevent oscillations and ensure a stable output common-mode level.

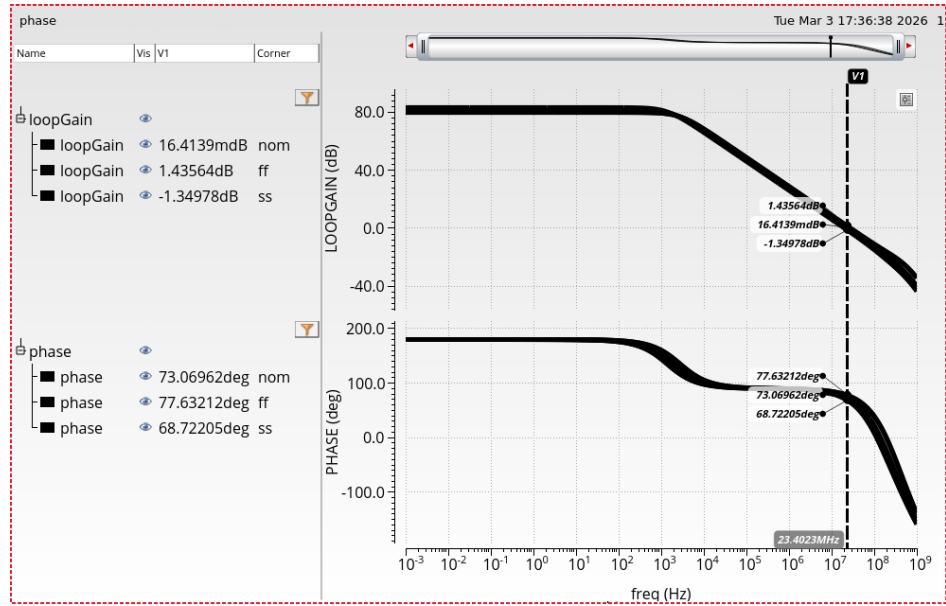


Figure 4.6. CMFB Loop Stability Simulation.

Common Mode Voltage. The output common-mode voltage is maintained by the CMFB loop. Figure 4.7 shows the Monte Carlo simulation results for the output common-mode voltage, demonstrating the effectiveness of the CMFB circuit across variations.

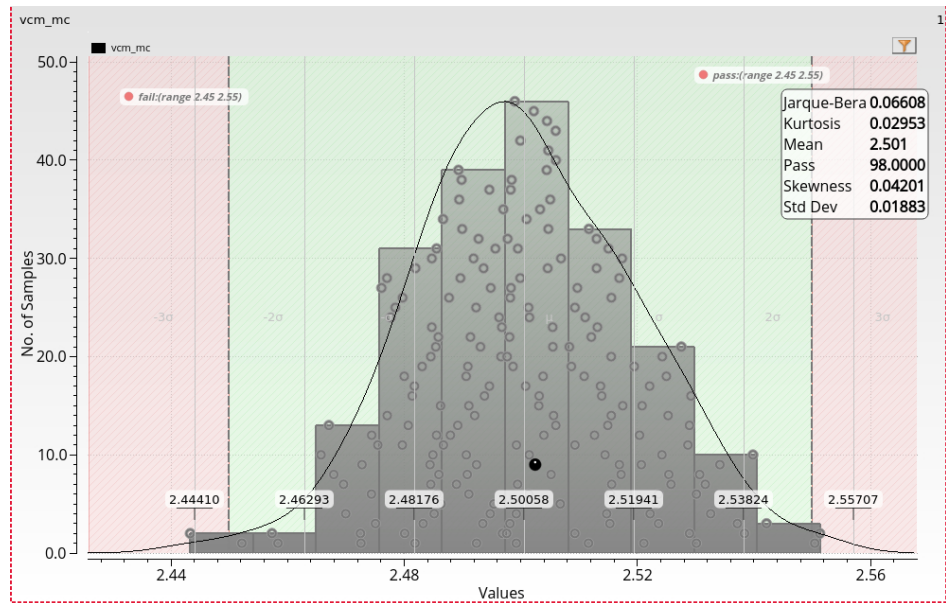


Figure 4.7. Output Common Mode Voltage Monte Carlo Simulation.

DIFF OPEN LOOP Analysis. The open-loop differential path is also analyzed, mainly to observe its phase delay and ensure it settles quickly without excessive ringing.

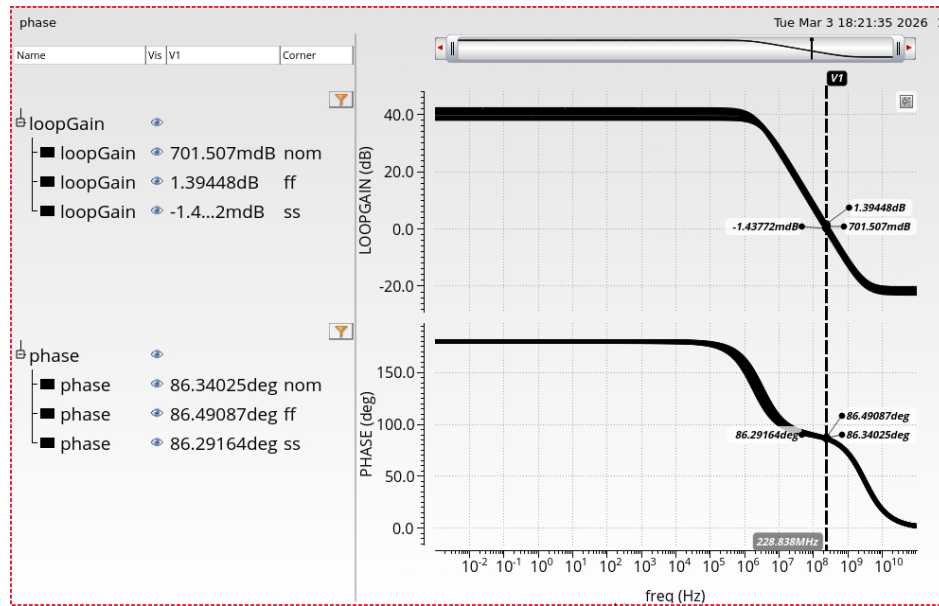


Figure 4.8. Differential Open Loop AC Response.

Offset. The input-referred offset directly impacts the effective resolution of the ADC.

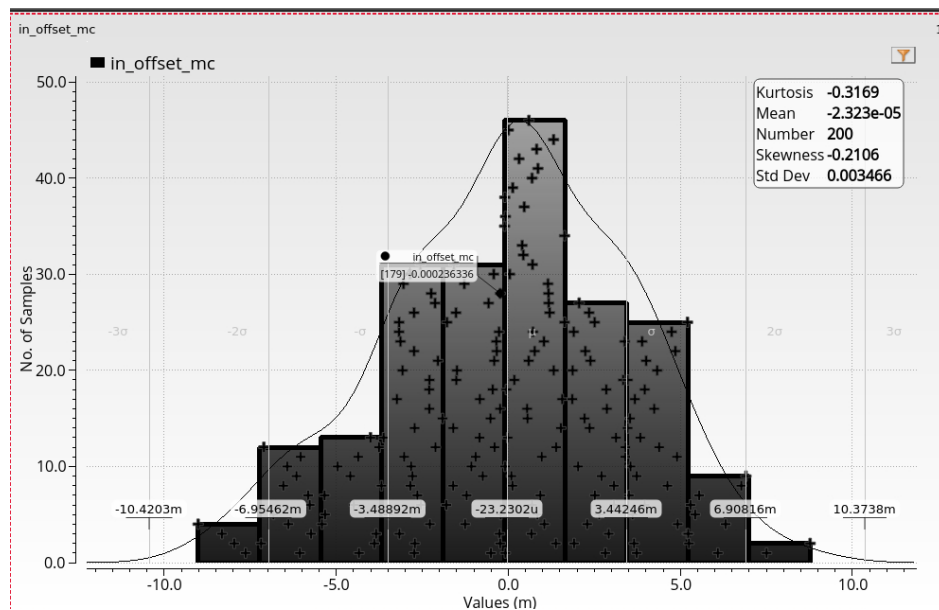


Figure 4.9. Input Referred Offset Monte Carlo Simulation.

Noise. The input-referred noise is another critical factor limiting the ADC's lsb resolution. The noise is integrated up to 37 MHz and calculated as $76.97 \mu V_{rms}$.

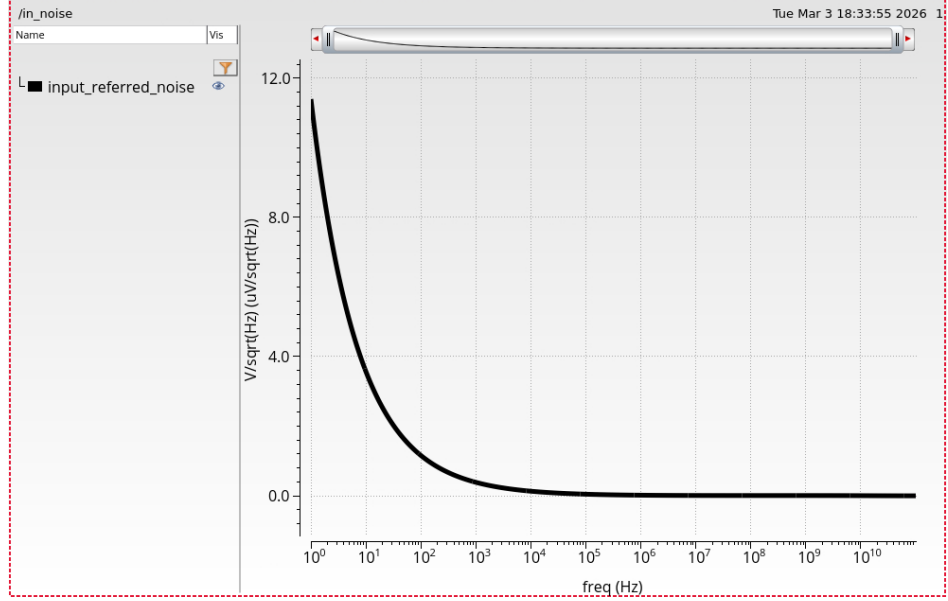


Figure 4.10. Input Referred Noise Simulation.

Table 4.2. Preamplifier Performance Summary

Parameter	Value
Differential Gain	38.7-41.8 dB
Differential Bandwidth (BW)	1.85-3.1 MHz
Differential GBW	228.8-269 MHz
CMFB Phase Margin	71.7-75.43°
CMFB GBW	20.1-27.6 MHz
Input Referred Offset (σ)	-+10 mV
Input Referred Noise (RMS)	76.97 μV
Output Common Mode Voltage (V_{ocm})	2.45-2.55 V

4.1.4. Comparator

The comparator is the decision-making block of the SAR ADC. To achieve high speed with zero static power consumption, a dynamic latch topology is employed. The design utilizes the StrongARM latch architecture, a widely adopted topology in

low-power ADCs. The implementation incorporates the insights and modifications presented by Razavi [8] to optimize the regeneration speed and input-referred offset.

4.1.4.1. Offset and Noise. Measuring the input-referred offset is straightforward using Monte Carlo simulations to find the trip point distribution. However, for noise performance, the dynamic nature of the latch requires a specific simulation strategy. The input-referred noise was analyzed using the transient noise simulation methodology described in [9].

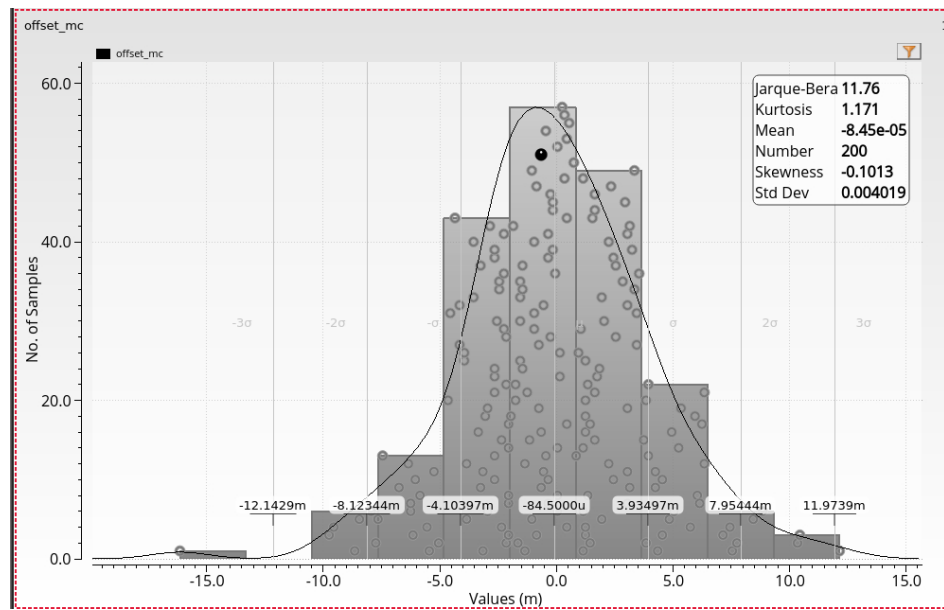


Figure 4.11. Comparator Input Referred Offset Monte Carlo Simulation.

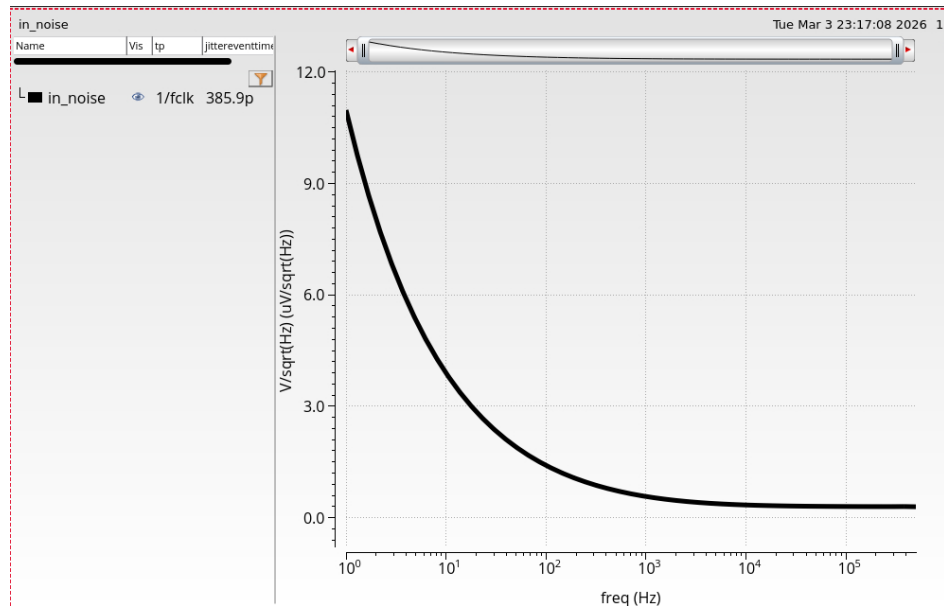


Figure 4.12. Comparator Input Referred Noise Simulation.

Table 4.3. Comparator Performance Summary

Parameter	Value
Input Referred Offset (σ)	-15-12 mV
Input Referred Integrated Noise (RMS)	216 μV

It is important to note that these values are referred to the input of the comparator. Since they are divided by the gain of the preamplifier when referred to the ADC input, their impact on the overall system performance is significantly reduced and considered negligible.

4.1.5. SAR Logic

The Successive Approximation Register (SAR) logic is the digital core that orchestrates the binary search algorithm. To maximize operation speed and avoid the need for a high-frequency oversampling clock, the logic is implemented using an asynchronous topology. The internal state transitions are self-timed, triggered by the ready signal from the comparator rather than a global clock edge.

The system operates across two voltage domains. The analog front-end, including the switches and capacitor array, operates at the 5V I/O voltage to maximize the signal-to-noise ratio. However, the complex digital logic is implemented using 1V core transistors to reduce dynamic power consumption and silicon area. Consequently, the comparator output is level-shifted down to 1V before being processed by the SAR engine. The resulting control signals are then level-shifted back up to 5V to drive the CapDAC switches.

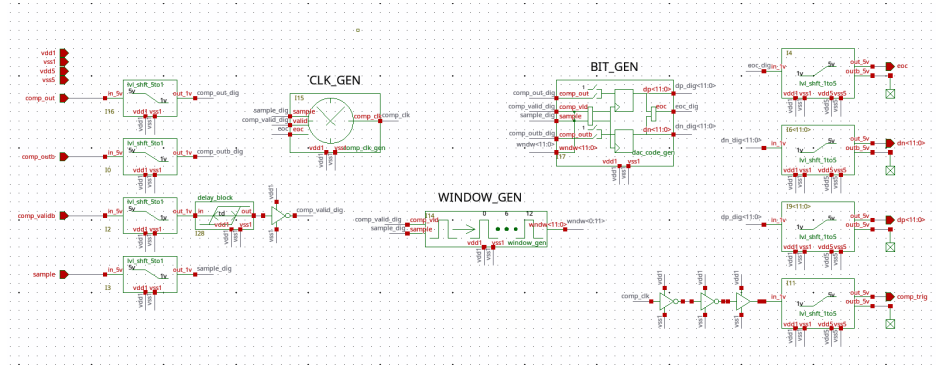


Figure 4.13. Asynchronous SAR Logic Overview with Level Shifters.

4.1.5.1. Window Generator. The window generator is responsible for orchestrating the timing of the bit trials. It generates 12 distinct window outputs, corresponding to the 12 bits of resolution. These signals are mutually exclusive; only one window bit is high at any given instance, indicating the specific bit currently being processed. This ensures the sequential operation of the SAR algorithm.

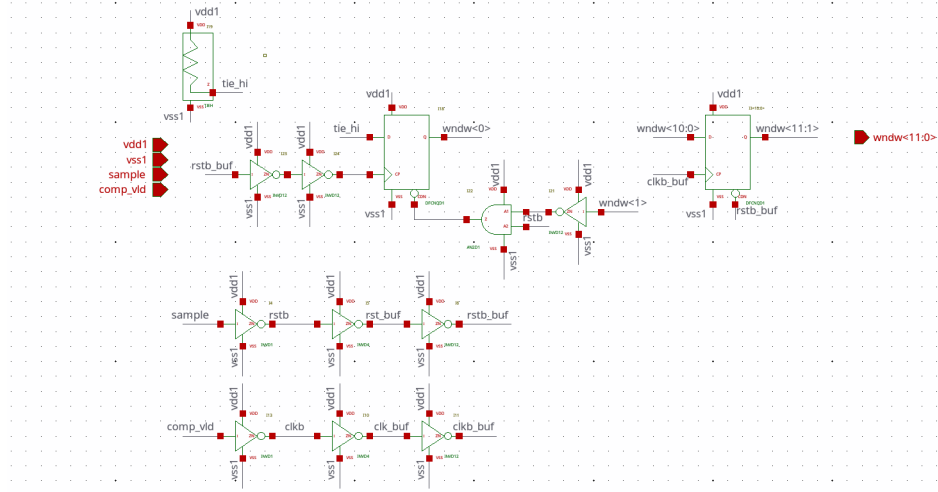


Figure 4.14. Window Generator Logic Implementation.

4.1.5.2. Comparator Clock Generator. The comparator clock is generated asynchronously to maximize speed and eliminate the need for a high-frequency external clock. The process initiates with the sampling signal, which triggers the first clock pulse. Subsequent clock pulses are self-timed; they are generated upon the receipt of a 'valid' signal from the comparator, indicating that a decision has been made. This loop continues, generating a new clock edge for each bit trial, until the End of Conversion (EOC) signal is asserted, which inhibits further clock generation. Ideally, the EOC signal should directly reset the flip-flops to minimize power consumption, but this optimization was not implemented.

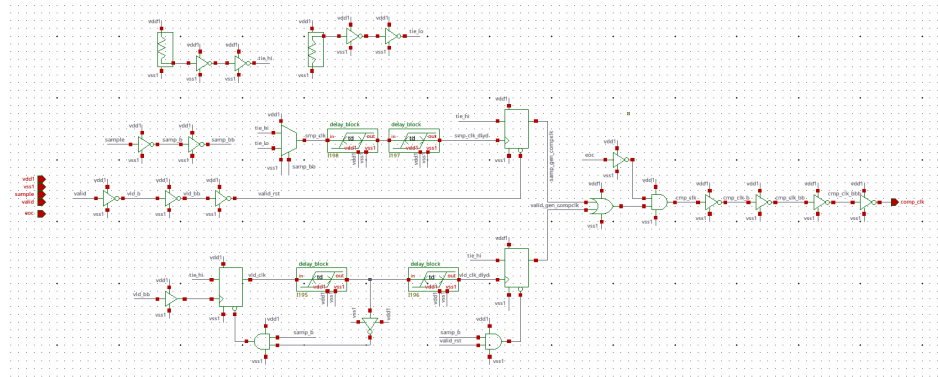


Figure 4.15. Asynchronous Comparator Clock Generation Logic.

A potential enhancement for this architecture is the implementation of adaptive

clock timing. Generally, the MSB capacitors in the DAC require a longer settling time compared to the LSB capacitors due to the larger voltage steps and stricter accuracy requirements. Conversely, the LSBs settle much faster. By leveraging the window bits, which track the current conversion step, the system can dynamically select different delay paths within the clock generation loop. This approach allocates more time for MSB decisions and speeds up LSB decisions, thereby optimizing the total conversion time. The bottom half of the circuit can be easily modified to implement this, but it is not necessary. To better illustrate the timing of the asynchronous operation, a simplified timing diagram is provided in Figure 4.16.

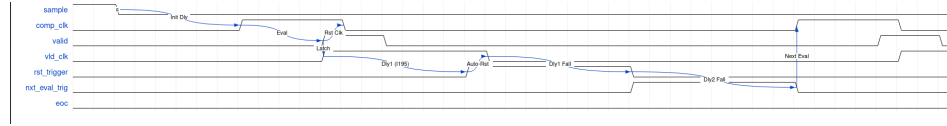


Figure 4.16. Timing Diagram of the Asynchronous Operation.

4.1.5.3. Bit Generator. The bit generator controls the sequential switching of the capacitor array, moving from the MSB to the LSB. There are no special design intricacies for this block; a direct set-and-down logic is implemented.

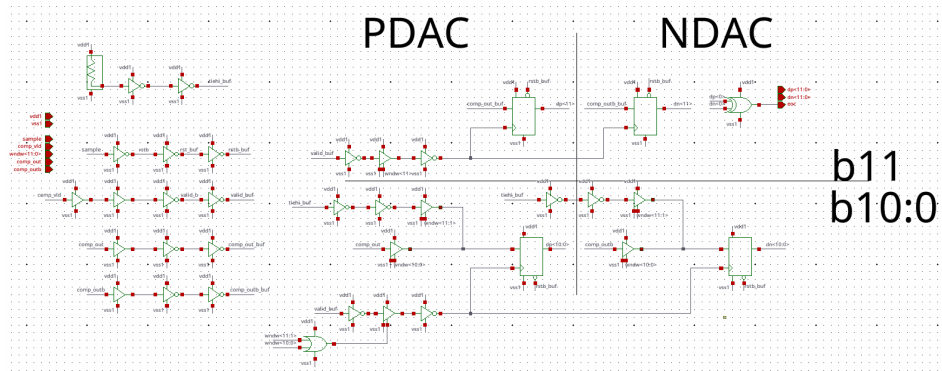


Figure 4.17. Bit Generator Logic Implementation.

4.1.6. ADC Buffer

To drive the switched-capacitor input of the SAR ADC and provide isolation from the preceding analog sensor stages, a dedicated ADC buffer is employed. The

buffer structure is designed based on an instrumentation amplifier topology to ensure high input impedance and precise differential signal driving.

The architecture consists of three amplifiers. The first stage utilizes two standard operational amplifiers configured as unity-gain buffers. These pre-amplifiers provide the necessary high input impedance so that the preceding sensor circuitry is not loaded. The outputs of these unity-gain buffers are then fed into the second stage, which employs a Fully Differential Amplifier (FDA). This third amplifier acts as a differential stage, converting the buffered signals into a robust, fully differential output specifically tailored to drive the CapDAC array of the SAR ADC while rejecting common-mode noise.

Figure 4.18 illustrates the schematic of the normal op-amp used in the unity-gain stage, and Figure 4.19 presents the schematic for the Fully Differential Amplifier. Additionally, Figure 4.20 shows the schematic for the overall instrumentation amplifier structure to demonstrate the differential topology.

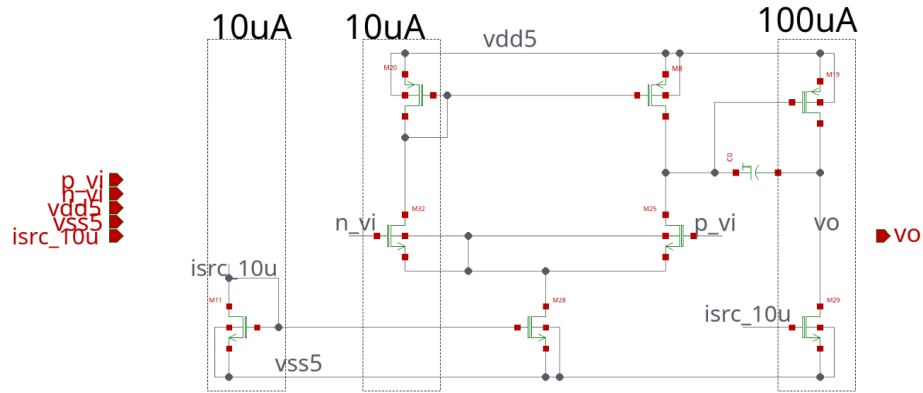


Figure 4.18. Schematic of the normal operational amplifier used in the unity-gain stage.

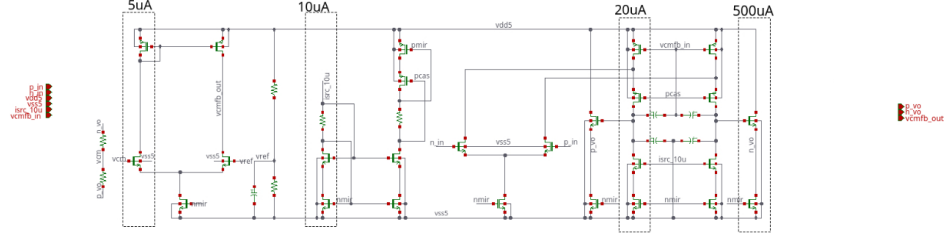


Figure 4.19. Schematic of the Fully Differential Amplifier (FDA) stage.

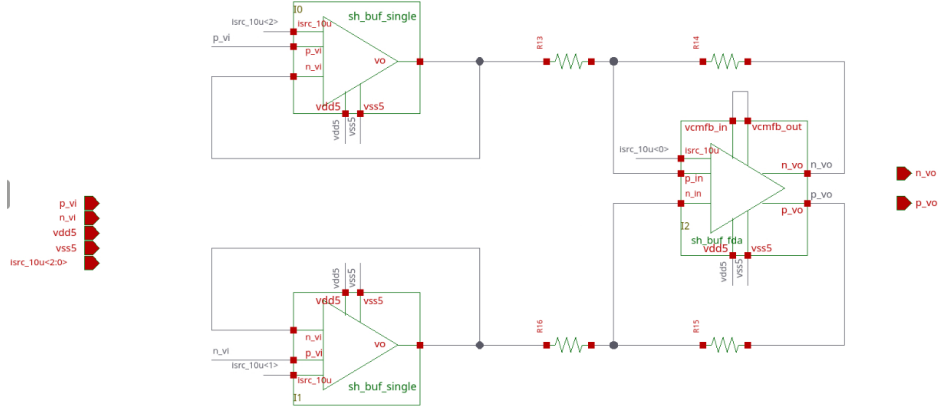


Figure 4.20. Schematic of the overall instrumentation amplifier structure.

To evaluate the dynamic performance of the ADC buffer, a testbench was created where the buffer is loaded by the Sample and Hold (S/H) switch described previously. The Signal-to-Noise and Distortion Ratio (SINAD) and Effective Number of Bits (ENOB) were measured. Due to the nature of the NMOS input stage used in the operational amplifiers, the input common-mode range is restricted, and the amplifiers do not operate linearly near the ground potential (typically under 1 V). As a result, the buffer maintains a SINAD corresponding to above 11.5 bits for approximately 70% of the input range, centered around a 2.5 V common-mode voltage. Consequently, the effective dynamic range of the buffer is limited to 70% of the Full Scale (FS) input.

4.1.7. LDO

To provide a clean, regulated reference voltage and isolate the sensitive analog blocks of the ADC from switching noise, a Low-Dropout (LDO) regulator is implemented. Figure 4.21 presents the schematic design of the LDO.

The primary design challenge for this LDO is maintaining stability across vastly different load conditions. During the ADC's operation, the load presented to the LDO changes dynamically. When the common-mode voltage (VCM) switch is disabled, the LDO is virtually unloaded, seeing only parasitic capacitances in the femtofarad (fF) range. However, when the VCM switch is enabled, the LDO connects to the top plates of the CapDAC array, which introduces a sudden capacitive load of approximately 10 pF.

Stability under these extreme load steps is achieved primarily by increasing the quiescent current in the output stage. Since the regulated output voltage is fixed at 2.5 V, there is sufficient voltage headroom to support this approach without degrading performance. While more advanced topologies, such as a Flipped Voltage Follower (FVF) architecture, could be employed to further enhance transient response and stability margins, the current design was deemed sufficient to meet the primary objectives and requirements of this thesis.

To ensure that the LDO remains stable and does not oscillate under these conditions, frequency response analyses were performed. Figure 4.22 illustrates the stability plot of the LDO, demonstrating adequate phase margin across the varying load conditions.

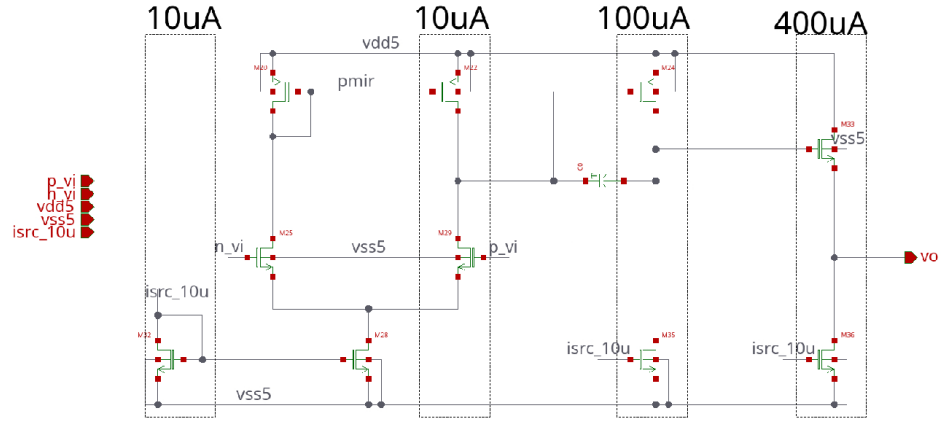


Figure 4.21. Schematic design of the Low-Dropout (LDO) regulator.

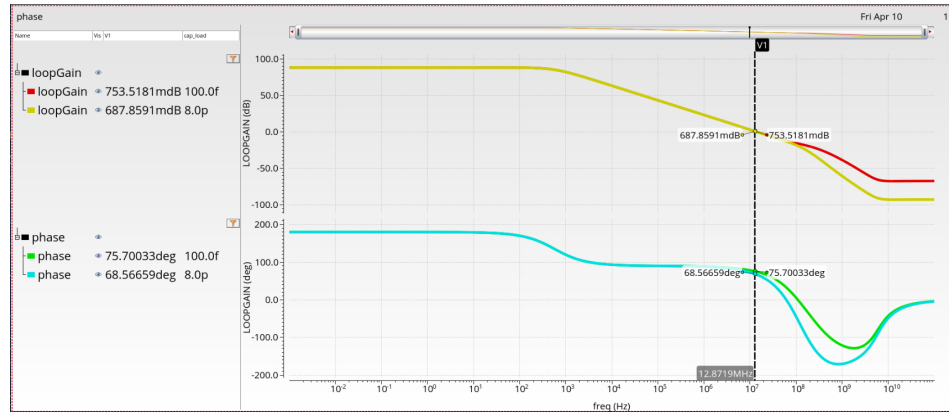


Figure 4.22. Stability plot of the LDO showing loop gain and phase margin.

4.1.8. ADC Top-Level Performance

The overall performance of the 12-bit SAR ADC was evaluated, considering the impact of noise, linearity, and process variations.

Given the 5V input range, the LSB size is approximately 1.22 mV. The combined input-referred noise from the preamplifier and comparator is significantly lower than the quantization noise level. Consequently, the system is not noise-limited, and the 12-bit resolution is not compromised by thermal noise.

The primary source of error is the Differential Non-Linearity (DNL) arising from the split-capacitor DAC array. As previously discussed, the DNL can approach ± 1

LSB. However, since the target Effective Number of Bits (ENOB) for this application is 10 bits, this linearity performance is considered acceptable. Nevertheless, as a precautionary measure to further minimize DNL errors, calibration capacitances were incorporated around the bridge capacitor.

Regarding the offset voltage introduced by the preamplifier and comparator, its impact on the MPPT algorithm is minimal. The system operates in a relatively slow-changing environment (solar irradiance and temperature), and the control logic relies primarily on the differential changes in voltage (dV) and current (dI) rather than absolute precision. Since the offset appears as a constant DC error across consecutive samples, it cancels out during the differential calculation, ensuring that the tracking direction remains accurate.

Figure 4.23 presents the 4096-point FFT of the ADC output, illustrating the dynamic performance. The analysis was performed using a Blackman-Harris window with a sampling frequency of 20 kSPS. The input frequency was chosen as $f_{in} = \frac{2039}{4096} f_s$, placing it close to the Nyquist frequency. The input signal was a differential sinusoid with a differential swing of ± 3.5 V (7 V peak-to-peak), as the maximum dynamic range is limited to 70% of the theoretical ± 5 V full scale by the ADC buffer. For the dynamic performance calculations, the peak saturation level was set to 5 V, and the first 10 harmonics were considered.

For the Slow-Slow (ss) and Fast-Fast (ff) process corners, a 256-point FFT was utilized. The primary purpose of this analysis was to verify the functional robustness of the asynchronous logic and to confirm that process variations do not cause timing issues, rather than to obtain high-precision dynamic performance metrics. Table 4.4 summarizes the key performance metrics across different process corners.

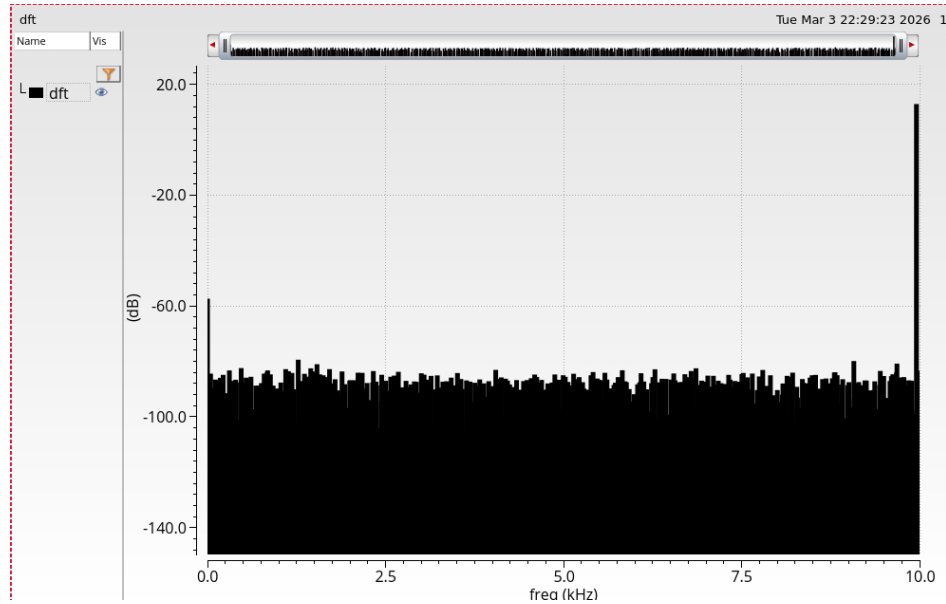


Figure 4.23. 4096-Point FFT of the ADC Output.

Table 4.4. ADC Performance Across Process Corners

Corner	SFDR (dB)	SNR (dB)	SINAD (dB)	ENOB (bits)
Typical (tt)	98.12	72.03	72	11.99
Slow-Slow (ss)	88.18	68.91	70.53	11.96
Fast-Fast (ff)	87.6	69.36	70.89	12.02

4.2. Duty Cycle Generator

The duty cycle generator is responsible for converting the digital control signal from the Fuzzy Logic Controller into a high-frequency Pulse Width Modulation (PWM) signal to drive the buck converter. This is achieved by comparing a stable analog reference voltage with a periodic triangular ramp signal using a high-speed comparator. As the analog reference voltage varies, the duty cycle of the resulting PWM signal changes proportionally. Figure 4.24 illustrates the top-level schematic of the duty cycle generator circuit.

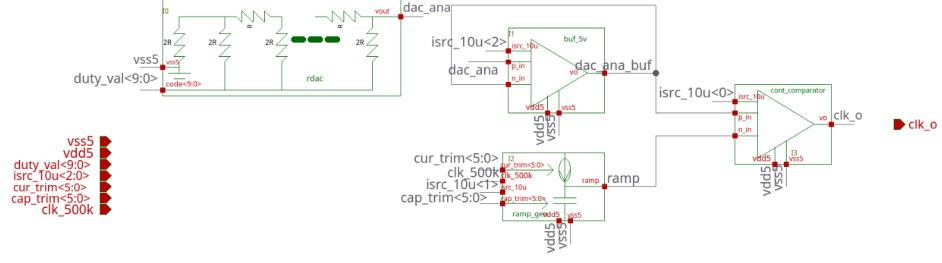


Figure 4.24. Schematic of the Duty Cycle Generator circuit.

4.2.1. DAC

To generate the precise analog reference voltage required by the duty cycle generator, an R-2R resistor ladder Digital-to-Analog Converter (DAC) is employed. The R-2R topology is chosen for its simplicity and relatively small silicon area footprint. The 10-bit digital output from the FLC controls the switches in the R-2R ladder, producing a proportional analog voltage.

The primary performance limitations in an R-2R DAC are Integral Non-Linearity (INL) and Differential Non-Linearity (DNL). These non-linearities arise predominantly from random and systematic mismatches in the resistors comprising the ladder. Since the accuracy of the analog output relies heavily on the precise 2 : 1 ratio of the resistance values, any process variations directly degrade the step sizes (DNL) and the overall transfer curve linearity (INL). To mitigate these mismatch errors and achieve an acceptable DNL, the unit resistor (R_{unit}) is designed to be relatively large. Specifically, P-poly type resistors are employed with a unit resistance of approximately 150 k Ω . Figure 4.25 presents the performance plot of the implemented 10-bit R-2R DAC, demonstrating its INL and DNL characteristics evaluated over a Monte Carlo simulation of 200 runs using a low-discrepancy sequence.

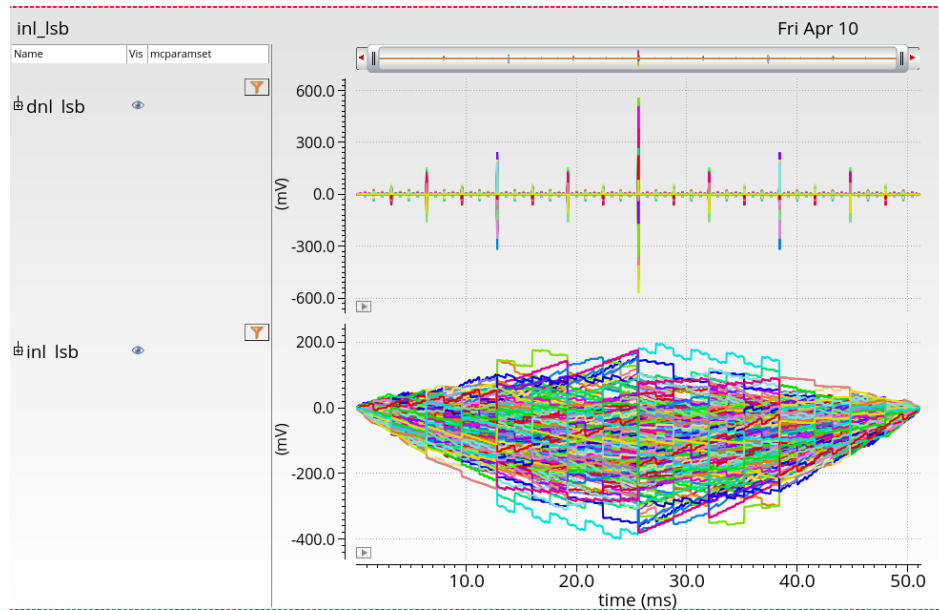


Figure 4.25. Performance plot showing the Integral Non-Linearity (INL) and Differential Non-Linearity (DNL) of the 10-bit R-2R DAC.

4.2.2. DAC Buffer

To prevent the subsequent high-speed comparator stage from loading the resistor network and distorting the reference voltage, a dedicated unity-gain operational amplifier buffer is placed at the output of the DAC. This buffer ensures that the reference voltage remains stable and accurate.

Unlike the amplifiers in the ADC buffer, the operational amplifier used for the DAC buffer employs a PMOS input stage. In a buck converter stepping down from a high PV panel voltage to a lower battery voltage, the Maximum Power Point (MPP) generally dictates a duty cycle lower than 0.5. Consequently, the analog reference voltage produced by the DAC predominantly remains in the lower half of the supply range. A PMOS input stage is specifically chosen for this application because it operates linearly and accurately even when the input common-mode voltage approaches the ground potential. Figure 4.26 illustrates the schematic of the DAC buffer.

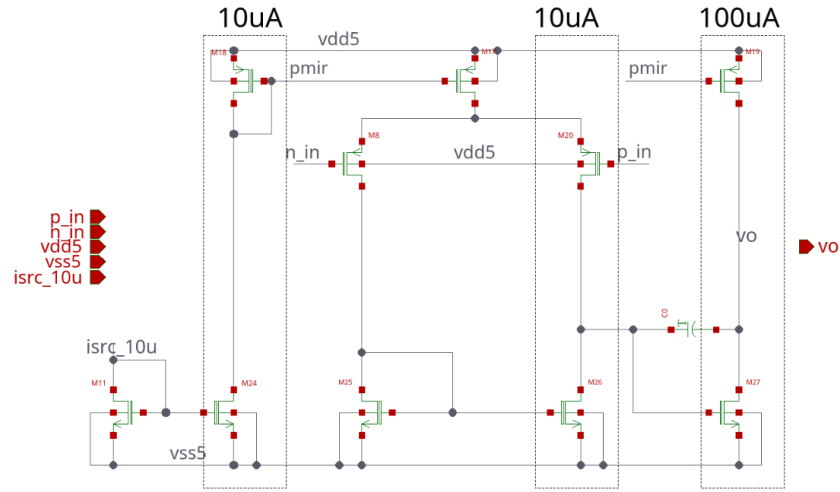


Figure 4.26. Schematic of the unity-gain operational amplifier with a PMOS input stage used for the DAC buffer.

Table 4.5 summarizes the key performance metrics of the designed operational amplifier used in the DAC buffer.

Table 4.5. DAC Buffer Amplifier Performance Summary

Parameter	Value
DC Gain	85.1 dB
Unity Gain Bandwidth (UGBW)	26.1 MHz
Phase Margin	81°
Input Referred Offset (σ)	+/-36 mV
Input Referred Noise (RMS)	850 μ V

4.2.3. Ramp Generator

A ramp generator circuit is utilized to produce a highly linear, periodic triangular waveform at the target switching frequency (500 kHz). This ramp signal serves as the time-base reference for the pulse width modulation process.

The core operating principle relies on charging a capacitor with a constant current

source. By pushing a fixed current into the capacitor, the voltage across it increases linearly over time. To create the periodic waveform, a switch is used to rapidly discharge (reset) the capacitor at every rising edge of the system clock, initiating a new ramp cycle.

To account for process variations and ensure the precise slope and amplitude of the ramp signal, both the charging current and the capacitor are adjustable. The capacitor array features a 6-bit binary-weighted trim network with a Least Significant Bit (LSB) capacitance of 125 fF. Similarly, the constant current source employs a 6-bit binary-weighted trim network with an LSB current of 1 μ A. Figure 4.27 illustrates the schematic of the ramp generator.

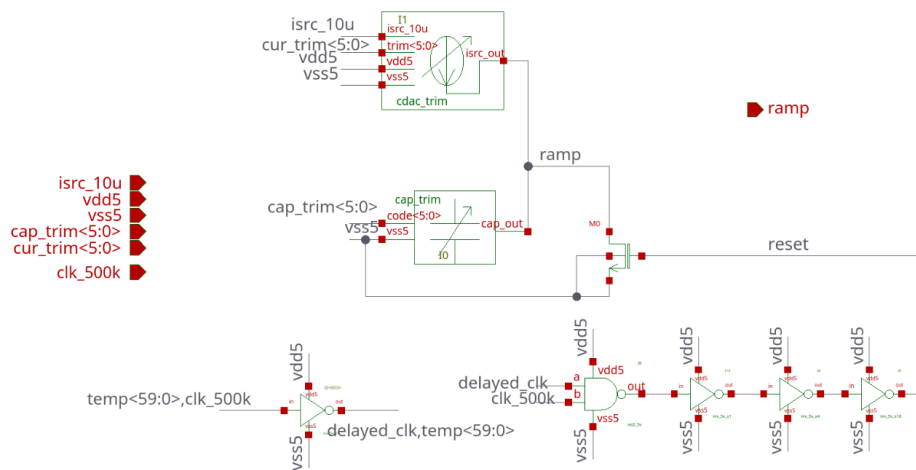


Figure 4.27. Schematic of the Ramp Generator circuit.

4.2.4. Continuous Comparator

The continuous-time high-speed comparator is the final stage of the duty cycle generator. It continuously compares the stable analog reference voltage from the DAC buffer against the high-frequency triangular wave from the ramp generator. For its implementation, an open-loop version of the operational amplifier used in the buffer stage is employed, with specific optimizations tailored for high-speed comparator operation. The output is a PWM signal whose duty cycle is directly proportional to the control voltage generated by the Fuzzy Logic Controller.

4.3. Bias and Bandgap Reference

To ensure stable operation across varying temperatures and supply voltages, the analog components of the system (including the ADC, DAC, and LDO) require robust bias currents and a stable reference voltage. A Bandgap Reference (BGR) circuit is utilized to fulfill these requirements. The detailed design and layout of the bandgap reference block were carried out by an undergraduate researcher, Yasin Öztürk, as part of a parallel project. Since the internal design of this block is outside the direct scope of this thesis, its specific implementation details are not discussed here. The author gratefully acknowledges his contribution to the overall system integration.

5. DIGITAL IMPLEMENTATION

5.1. Hardware-Friendly Fuzzy Logic Architecture

While the theoretical model of the Fuzzy Logic Controller (FLC) was successfully validated in MATLAB and SystemVerilog, a direct translation of these high-level algorithms into physical digital logic is highly inefficient. Standard fuzzy logic operations—specifically the linear interpolation required during fuzzification and the weighted average calculation used in Sugeno defuzzification—rely heavily on complex arithmetic operations such as division and multiplication.

In digital hardware design, true multipliers and dividers are "logic-hungry" components; they consume a significant amount of silicon area, increase dynamic power consumption, and introduce long critical path delays. This makes a direct mathematical implementation unsuitable for a low-cost, highly integrated Application-Specific Integrated Circuit (ASIC).

To bridge the gap between theoretical performance and practical physical implementation, this chapter presents a highly optimized, hardware-friendly architecture. The core principle of this optimization is the elimination of complex arithmetic blocks. Instead, the fuzzy logic algorithms are reformulated to utilize highly efficient bitwise shift operations, simple adders, and compact Look-Up Tables (LUTs). The following subsections detail how these architectural simplifications are applied to each stage of the fuzzy inference process without significantly degrading the overall Maximum Power Point Tracking (MPPT) performance.

5.1.1. Difference Calculation

The first step in the MPPT algorithm is determining the change in the photovoltaic voltage (dV) and the change in output current (dI) between consecutive sam-

pling intervals. In a digital hardware implementation, this requires storing the previous states using D-type flip-flop registers. At each clock cycle, a simple digital subtractor computes the difference between the current and previous samples, as shown in Code Snippet 5.1.

Code Snippet 5.1. Difference Generator SystemVerilog Implementation

```

module diff_generator (
    input  logic      clk,
    input  logic      rst_n,
    input  logic [9:0] in_value,      // current input sample
    output logic signed [10:0] diff   // signed difference
        output
);

    logic [9:0] prev_value;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            prev_value <= 10'd0;
        else
            prev_value <= in_value;
    end

    // Difference between current and previous sample
    always_comb begin
        diff = $signed({1'b0, in_value}) - $signed({1'b0,
            prev_value});
    end

endmodule

```

5.1.2. Fuzzification

The fuzzification stage assigns membership values to fuzzify the crisp digital inputs into linguistic variables. An exact calculation of the membership degree (typically ranging from 0 to 255 for an 8-bit resolution) involves linear interpolation, which inherently requires a division operation. To circumvent this, the interpolation fraction is optimized by lookup dividing logic. The denominator is approximated to the nearest power of two, which transforms the complex division into a highly efficient bitwise right-shift ($\gg$) operation. This replaces a resource-intensive divider with simple comparative lookup logic and shift operations.

The hardware implementation utilizes three primary types of membership functions: Z-shaped, S-shaped, and Triangular-shaped.

5.1.2.1. Z-Shaped Membership Function. The Z-shaped membership function implements a falling linear ramp. It assigns a high membership degree when the input is below a starting threshold (A) and a low membership degree when it exceeds the end threshold (B). For intermediate values, it computes the declining slope using the optimized lookup dividing logic. The SystemVerilog code is provided in [Code Snippet 5.2](#).

Code Snippet 5.2. Z-Shaped Membership Function SystemVerilog Implementation

```
module mf_z (
    input  logic signed [10:0] A,
    input  logic signed [10:0] B,
    input  logic signed [10:0] x,
    output logic [7:0] mu

);

    logic signed [11:0] diff, denom;
    logic signed [19:0] scaled;
    logic [7:0] shifted_result;
```

```

logic [4:0] shift_amount;

always_comb begin
    if (x <= A) begin
        mu = 8'd255;          // High at left
    end
    else if (x >= B) begin
        mu = 8'd0;           // Low at right
    end
    else begin
        // Falling linear ramp:  $\mu = 255 * (B - x) / (B - A)$ 
        diff = B - x;
        denom = B - A;

        // Determine shift amount based on CLOSEST power
        // of 2 for minimal quantization error
        if (denom < 12'd12) shift_amount = 5'd3;    //  $2^3$ 
            = 8
        else if (denom < 12'd24) shift_amount = 5'd4;
            //  $2^4 = 16$ 
        else if (denom < 12'd48) shift_amount = 5'd5;
            //  $2^5 = 32$ 
        else if (denom < 12'd96) shift_amount = 5'd6;
            //  $2^6 = 64$ 
        else if (denom < 12'd192) shift_amount = 5'd7;
            //  $2^7 = 128$ 
        else if (denom < 12'd384) shift_amount = 5'd8;
            //  $2^8 = 256$ 
        else if (denom < 12'd768) shift_amount = 5'd9;
            //  $2^9 = 512$ 
        else if (denom < 12'd1536) shift_amount = 5'd10;
            //  $2^{10} = 1024$ 
    end
end

```

```

        else shift_amount = 5'd11; // 2^11 = 2048 (
            standard 10-bit ADC range)

        // Hardware-efficient: multiply and right-shift
            instead of divide
        // mu = 255 * (B - x) / (B - A) ~ (255 * diff) >>
            shift_amount
        scaled = diff * 8'd255;
        shifted_result = scaled >> shift_amount;

        mu = (shifted_result > 255) ? 8'd255 :
            shifted_result;
    end
end
endmodule

```

5.1.2.2. S-Shaped Membership Function. The S-shaped membership function corresponds to a rising linear ramp. It evaluates to 0 for inputs below A and reaches its maximum value of 255 for inputs above B . The linear increase between the thresholds is computed using the same hardware-efficient shift approximation. The SystemVerilog implementation is shown in Code Snippet 5.3.

Code Snippet 5.3. S-Shaped Membership Function SystemVerilog Implementation

```

module mf_s (
    input  logic signed [10:0] A,
    input  logic signed [10:0] B,
    input  logic signed [10:0] x,
    output logic [7:0] mu

);

    logic signed [11:0] diff, denom;
    logic signed [19:0] scaled;

```

```

logic [7:0] shifted_result;
logic [4:0] shift_amount;

always_comb begin
    if (x <= A) begin
        mu = 8'd0;
    end
    else if (x >= B) begin
        mu = 8'd255;
    end
    else begin
        // Rising linear ramp:  $\mu = 255 * (x - A) / (B - A)$ 
        diff = x - A;
        denom = B - A;

        // Determine shift amount based on CLOSEST power
        // of 2 for minimal quantization error
        if (denom < 12'd12) shift_amount = 5'd3;    //  $2^3$ 
            = 8
        else if (denom < 12'd24) shift_amount = 5'd4;
            //  $2^4 = 16$ 
        else if (denom < 12'd48) shift_amount = 5'd5;
            //  $2^5 = 32$ 
        else if (denom < 12'd96) shift_amount = 5'd6;
            //  $2^6 = 64$ 
        else if (denom < 12'd192) shift_amount = 5'd7;
            //  $2^7 = 128$ 
        else if (denom < 12'd384) shift_amount = 5'd8;
            //  $2^8 = 256$ 
        else if (denom < 12'd768) shift_amount = 5'd9;
            //  $2^9 = 512$ 
    end
end

```

```

else if (denom < 12'd1536) shift_amount = 5'd10;
    // 2^10 = 1024
else shift_amount = 5'd11; // 2^11 = 2048 (
    standard 10-bit ADC range)

// Hardware-efficient: multiply and right-shift
    instead of divide
// mu = 255 * (x - A) / (B - A) ~ (255 * diff) >>
    shift_amount
scaled = diff * 8'd255;
shifted_result = scaled >> shift_amount;

mu = (shifted_result > 255) ? 8'd255 :
    shifted_result;
end
end
endmodule

```

5.1.2.3. Triangular-Shaped Membership Function. The triangular membership function specifies a peak response. It combines a rising ramp from an initial point A to a peak at B , and a falling ramp from B to an end point C . The division for interpolating both slopes relies on the shift-amount lookup table to minimize quantization error. Its SystemVerilog code is provided in Code Snippet 5.4.

Code Snippet 5.4. Triangular-Shaped Membership Function SystemVerilog Implementation

```

module mf_triangle (
    input  logic signed [10:0] A,    // Left slope start
    input  logic signed [10:0] B,    // Peak
    input  logic signed [10:0] C,    // Right slope end
    input  logic signed [10:0] x,    // input variable (ADC
    diff)

```



```

output logic [7:0] mu          // membership degree
    (0-255)

);

// Internal signals
logic signed [11:0] num1, num2, den1, den2;
logic signed [19:0] scaled;
logic [7:0] shifted_result;
logic [4:0] shift_amount_left, shift_amount_right;

always_comb begin
    // Default outputs
    mu = 8'd0;

    // Left slope: A < x < B (rising)
    if ((x > A) && (x < B)) begin
        num1 = x - A;
        den1 = B - A;

        // Determine shift amount based on CLOSEST power
        // of 2
        if (den1 < 12'd12) shift_amount_left = 5'd3;    //
            2^3 = 8
        else if (den1 < 12'd24) shift_amount_left = 5'd4;
            // 2^4 = 16
        else if (den1 < 12'd48) shift_amount_left = 5'd5;
            // 2^5 = 32
        else if (den1 < 12'd96) shift_amount_left = 5'd6;
            // 2^6 = 64
        else if (den1 < 12'd192) shift_amount_left = 5'd7
            ;    // 2^7 = 128
        else if (den1 < 12'd384) shift_amount_left = 5'd8
            ;    // 2^8 = 256
    end
end

```

```

else if (den1 < 12'd768) shift_amount_left = 5'd9
    ;    // 2^9 = 512
else if (den1 < 12'd1536) shift_amount_left = 5'
    d10;    // 2^10 = 1024
else shift_amount_left = 5'd11;    // 2^11 = 2048

// Hardware-efficient: mu = 255 * (x - A) / (B -
    A) ~ (255 * num1) >> shift_amount
scaled = num1 * 8'd255;
shifted_result = scaled >> shift_amount_left;
mu = (shifted_result > 255) ? 8'd255 :
    shifted_result;
end

// Right slope: B <= x < C (falling)
else if ((x >= B) && (x < C)) begin
    num2 = C - x;
    den2 = C - B;

    // Determine shift amount based on CLOSEST power
    of 2
    if (den2 < 12'd12) shift_amount_right = 5'd3;
        // 2^3 = 8
    else if (den2 < 12'd24) shift_amount_right = 5'd4
        ;    // 2^4 = 16
    else if (den2 < 12'd48) shift_amount_right = 5'd5
        ;    // 2^5 = 32
    else if (den2 < 12'd96) shift_amount_right = 5'd6
        ;    // 2^6 = 64
    else if (den2 < 12'd192) shift_amount_right = 5'
        d7;    // 2^7 = 128
    else if (den2 < 12'd384) shift_amount_right = 5'
        d8;    // 2^8 = 256

```

```

        else if (den2 < 12'd768) shift_amount_right = 5'd9;    // 2^9 = 512
        else if (den2 < 12'd1536) shift_amount_right = 5'd10; // 2^10 = 1024
        else shift_amount_right = 5'd11; // 2^11 = 2048

        // Hardware-efficient: mu = 255 * (C - x) / (C - B) ~ (255 * num2) >> shift_amount
        scaled = num2 * 8'd255;
        shifted_result = scaled >> shift_amount_right;
        mu = (shifted_result > 255) ? 8'd255 :
            shifted_result;
    end
    // Peak: x == B
    else if (x == B) begin
        mu = 8'd255;
    end
end

endmodule

```

5.1.2.4. Fuzzifier Module. The fuzzifier module acts as a top-level covering wrapper that incorporates the individual Z-shaped, S-shaped, and Triangular-shaped membership functions to build the complete three-input fuzzification stage. It instantiates these hardware-efficient blocks to evaluate the three primary inputs: the change in PV voltage (dV), the change in output current (dI), and the absolute PV voltage (V_{pv}). By operating these instances in parallel, the module rapidly assigns the corresponding linguistic membership degrees required by the subsequent inference engine. The SystemVerilog implementation for this encompassing module is provided in Code Snippet 5.5.

Code Snippet 5.5. Fuzzifier System Verilog Implementation

```

module fuzzifier(
    input  logic signed [10:0] diff_V,    // dV_raw (
        derivative of voltage)
    input  logic signed [10:0] diff_I,    // dI_raw (
        derivative of current)
    input  logic signed [10:0] V_pv,      // V_pv differential
        (ADC: Vpv/12 - 2.5V, center at 512)

    // dV Membership Function Parameters
    input  logic signed [10:0] dV_A_neg, dV_B_neg,
    input  logic signed [10:0] dV_A_tri, dV_B_tri, dV_C_tri,
    input  logic signed [10:0] dV_A_pos, dV_B_pos,

    // dI Membership Function Parameters
    input  logic signed [10:0] dI_A_neg, dI_B_neg,
    input  logic signed [10:0] dI_A_tri, dI_B_tri, dI_C_tri,
    input  logic signed [10:0] dI_A_pos, dI_B_pos,

    // Vpv Membership Function Parameters
    input  logic signed [10:0] Vpv_A_low, Vpv_B_low,
    input  logic signed [10:0] Vpv_A_opt, Vpv_B_opt,
        Vpv_C_opt,
    input  logic signed [10:0] Vpv_A_high, Vpv_B_high,

    // dV Membership degrees
    output logic [7:0] dV_mu_neg,
    output logic [7:0] dV_mu_ze,
    output logic [7:0] dV_mu_pos,

    // dI Membership degrees
    output logic [7:0] dI_mu_neg,
    output logic [7:0] dI_mu_ze,

```

```

output logic [7:0] dI_mu_pos,

// V_pv Membership degrees
output logic [7:0] Vpv_mu_low,
output logic [7:0] Vpv_mu_opt,
output logic [7:0] Vpv_mu_high
);

//=====
// dV_raw Fuzzification (Match: [-5 0 5] ranges)
//=====
// Negative: linzmf
mf_z      mf_dv_neg (.A(dV_A_neg), .B(dV_B_neg), .x(
    diff_V), .mu(dV_mu_neg));
// Zero: trimf
mf_triangle mf_dv_ze (.A(dV_A_tri), .B(dV_B_tri), .C(
    dV_C_tri), .x(diff_V), .mu(dV_mu_ze));
// Positive: linsmf
mf_s      mf_dv_pos (.A(dV_A_pos), .B(dV_B_pos), .x(
    diff_V), .mu(dV_mu_pos));

//=====
// dI_raw Fuzzification (Match: [-5 0 5] ranges)
//=====
// Negative: linzmf
mf_z      mf_di_neg (.A(dI_A_neg), .B(dI_B_neg), .x(
    diff_I), .mu(dI_mu_neg));
// Zero: trimf
mf_triangle mf_di_ze (.A(dI_A_tri), .B(dI_B_tri), .C(
    dI_C_tri), .x(diff_I), .mu(dI_mu_ze));
// Positive: linsmf
mf_s      mf_di_pos (.A(dI_A_pos), .B(dI_B_pos), .x(
    diff_I), .mu(dI_mu_pos));

```

```

//=====
// V_pv Fuzzification (DIFFERENTIAL INPUT, Vcm at 512)
// 12-bit ADC, top 10 bits used (1 LSB = 5mV * 12 = 60mV)
// Scaling: Vpv_analog / 12, compared against 2.5V (Vcm)
// MPP operating window: 28V-32V (narrow band around 30V
// MPP)
// ADC center (30V): 512
//=====

// 1. Low_Voltage (Turbo Mode)
mf_z          mf_vpv_low  (.A(Vpv_A_low), .B(Vpv_B_low), .x
    (V_pv), .mu(Vpv_mu_low));

// 2. Optimal_Range (The Sweet Spot)
mf_triangle mf_vpv_opt  (.A(Vpv_A_opt), .B(Vpv_B_opt), .C
    (Vpv_C_opt), .x(V_pv), .mu(Vpv_mu_opt));

// 3. High_Voltage (Safety Brake)
mf_s          mf_vpv_high (.A(Vpv_A_high), .B(Vpv_B_high),
    .x(V_pv), .mu(Vpv_mu_high));

endmodule

```

5.1.3. Inference Engine

The inference phase continuously evaluates the IF-THEN rules based on the three linguistic input variables: dV , dI , and absolute V_{pv} . A standard three-input fuzzy system with three linguistic states each would require a full combinatorial rule base of 27 distinct rules ($3 \times 3 \times 3$). To significantly reduce computational complexity and silicon area, the proposed architecture streamlines this rule base. The absolute PV voltage (V_{pv}) acts as a primary safety parameter; when it is operating far from the MPP volt-

age range, it overrides the other inputs using only two boundary rules. When within the optimal range, only 9 rules based on dV and dI are evaluated. This structural reduction from 27 to 11 total rules dramatically reduces the size of the required combinational logic and look-up tables. As shown in Code Snippet 5.6, the SystemVerilog implementation further consolidates these 11 conceptual rules into just 8 explicitly calculated hardware weights by grouping the stable "Hold" conditions together. A custom 3-input minimum function continuously evaluates the rule firing strengths using purely combinational logic.

Code Snippet 5.6. Inference Engine SystemVerilog Implementation

```

module inference (
    // dV and dI inputs (3 each)
    input  logic [7:0] dV_mu_neg, dV_mu_ze, dV_mu_pos,
    input  logic [7:0] dI_mu_neg, dI_mu_ze, dI_mu_pos,
    // V_pv inputs (3)
    input  logic [7:0] Vpv_mu_low, Vpv_mu_opt, Vpv_mu_high,

    // Rule Firing Strengths (w1 through w8)
    output logic [7:0] w [1:8]
);

// Internal function for 3-input MIN
function automatic logic [7:0] min3(logic [7:0] a, logic
[7:0] b, logic [7:0] c);
    logic [7:0] m2;
    m2 = (a < b) ? a : b;
    return (m2 < c) ? m2 : c;
endfunction

always_comb begin
    // --- GROUP A: SAFETY RULES ---
    // Rule 1: V=Low -> Big_Decrease (w1)
    w[1] = Vpv_mu_low;

```

```

// Rule 2: V=High -> Big_Increase (w2)
w[2] = Vpv_mu_high;

// --- GROUP B: FINE TUNING (Only active in Optimal
// Range) ---
// Rule 3: Pos/Pos/Opt -> Decrease
w[3] = min3(dV_mu_pos, dI_mu_pos, Vpv_mu_opt);

// Rule 4: Pos/Neg/Opt -> Increase
w[4] = min3(dV_mu_pos, dI_mu_neg, Vpv_mu_opt);

// Rule 5: Neg/Pos/Opt -> Increase
w[5] = min3(dV_mu_neg, dI_mu_pos, Vpv_mu_opt);

// Rule 6: Neg/Neg/Opt -> Decrease
w[6] = min3(dV_mu_neg, dI_mu_neg, Vpv_mu_opt);

// Rule 7: Stuck (Ze/Ze/Opt) -> HOLD
w[7] = min3(dV_mu_ze, dI_mu_ze, Vpv_mu_opt);

// Rule 8: Stable (Ze/Any/Opt) -> HOLD (Combining
// your two stable rules)
// We can simplify the "Stable" logic here or list
// them separately.
w[8] = min3(dV_mu_ze, (dI_mu_neg > dI_mu_pos ?
    dI_mu_neg : dI_mu_pos), Vpv_mu_opt);

end
endmodule

```


5.1.4. Defuzzification

The final control output, ΔD , is obtained using a Sugeno-type weighted average:

$$\Delta D = \frac{\sum_{i=1}^n w_i \cdot c_i}{\sum_{i=1}^n w_i} \quad (5.1)$$

In hardware, the multiplication ($w_i \cdot c_i$) and the division by the sum of weights are computationally expensive. We optimize this by relying on the synthesizer to implement efficient constant multiplications (or shifts and adders) for the fixed crisp output constants (C_i). More importantly, the hardware-expensive division by the denominator summation ($\sum w_i$) is completely eliminated. By identifying the nearest power of two to the total weight, the division is transformed into an efficient bitwise right-shift ($\gg$) operation. This approximation drastically reduces the logic footprint while maintaining adequate precision for the duty cycle adjustment. The SystemVerilog implementation of this optimized defuzzifier is presented in Code Snippet 5.7.

Code Snippet 5.7. Defuzzifier SystemVerilog Implementation

```

module defuzzifier (
    input  logic [7:0] w [1:8],                // 8 Rule
        weights from Inference
    input  logic signed [7:0] C1, C2, C3, C4,    // Sugeno
        constants (tunable)
    input  logic signed [7:0] C5, C6, C7, C8,
    output logic signed [10:0] dDuty             // Final
        step size
);

    logic signed [20:0] sum_wi_ci;
    logic [9:0] sum_wi;
    logic [4:0] shift_amount;

    always_comb begin

```

```

//=====

// WEIGHTED SUM:  $\sum(w_i * C_i)$ 
//=====

sum_wi_ci = ( $signed({1'b0, w[1]}) * C1 ) +
              ( $signed({1'b0, w[2]}) * C2 ) +
              ( $signed({1'b0, w[3]}) * C3 ) +
              ( $signed({1'b0, w[4]}) * C4 ) +
              ( $signed({1'b0, w[5]}) * C5 ) +
              ( $signed({1'b0, w[6]}) * C6 ) +
              ( $signed({1'b0, w[7]}) * C7 ) +
              ( $signed({1'b0, w[8]}) * C8 );

//=====

// TOTAL WEIGHT:  $\sum(w_i)$ 
//=====

sum_wi = {1'b0, w[1]} + {1'b0, w[2]} + {1'b0, w[3]} +
          {1'b0, w[4]} +
          {1'b0, w[5]} + {1'b0, w[6]} + {1'b0, w[7]} +
          {1'b0, w[8]};

//=====

// HARDWARE-EFFICIENT DEFUZZIFICATION: Replace
// Divider with Right-Shift
// Find the CLOSEST power of 2 to sum_wi for minimal
// quantization error
//=====

if (sum_wi == 0) begin

```

```

        shift_amount = 5'd0; // Avoid division by zero
    end
    else if (sum_wi < 12'd24) begin
        // For very small sums, use shift_amount = 4 (2^4
        // = 16)
        shift_amount = 5'd4;
    end
    else if (sum_wi < 12'd48) begin
        shift_amount = 5'd5; // 2^5 = 32
    end
    else if (sum_wi < 12'd96) begin
        shift_amount = 5'd6; // 2^6 = 64
    end
    else if (sum_wi < 12'd192) begin
        shift_amount = 5'd7; // 2^7 = 128
    end
    else if (sum_wi < 12'd384) begin
        shift_amount = 5'd8; // 2^8 = 256
    end
    else if (sum_wi < 12'd768) begin
        shift_amount = 5'd9; // 2^9 = 512
    end
    else if (sum_wi < 12'd1536) begin
        shift_amount = 5'd10; // 2^10 = 1024
    end
    else begin
        shift_amount = 5'd11; // 2^11 = 2048 (max case)
    end

    // Sugeno defuzzification:  $\Delta D \approx (\sum(wi * Ci)) >>$ 
    shift_amount

    // Now uses closest power-of-2 for much better
    accuracy

```

```

        if (sum_wi != 0) begin
            dDuty = sum_wi_ci >> shift_amount;
        end else begin
            dDuty = 11'sd0;
        end
    end
end
endmodule

```

5.1.5. Duty Integration

The defuzzified output represents the necessary *change* in the duty cycle (ΔD). The final stage is the discrete-time integration of this output to generate the absolute duty cycle for the PWM block. This is implemented using a digital accumulator.

To degrade further oscillations and provide a digital damping effect, the system accumulates these integer steps into a higher-resolution 26-bit duty cycle register, but only slices the top 10 bits for the final DAC output. This fractional accumulation dampens the instantaneous adjustments and compensates for any quantization noise introduced by the approximations in the preceding fuzzification and defuzzification stages.

Furthermore, the module handles transitions between manual control and active MPPT tracking by pre-loading the accumulator with the scaled manual duty cycle. When the FLC takes over, it resumes accumulation from this exact state. This mechanism guarantees a "bumpless" transfer without abrupt adjustments, maintaining stability around the Maximum Power Point. The SystemVerilog implementation of the duty integrator, which also incorporates boundary saturation logic to ensure safe operation limits, is provided in Code Snippet 5.8.

Code Snippet 5.8. Duty Integrator SystemVerilog Implementation

```

module duty_integrator (
    input  logic          clk,

```

```

input  logic      reset_n,
input  logic      mppt_enable,
input  logic signed [10:0] duty_adjust,
input  logic [9:0] manual_duty_in,
output logic [9:0] dac_value

);

// --- CONFIGURATION ---
// CHANGE THIS: 4 for Turbo Sim, 14 for Real Hardware
localparam int FRAC_BITS = 4;

// Safety Headroom: 10 (Data) + FRAC_BITS + 2 (Sign/
// Overflow)
// Keep 26 bits because it is safe for both 9 and 14.
logic signed [25:0] acc;

// --- SCALED LIMITS (Calculated automatically) ---
localparam signed [25:0] MIN_ACC = 26'd307 << FRAC_BITS;
localparam signed [25:0] MAX_ACC = 26'd716 << FRAC_BITS;
localparam signed [25:0] START_VAL = 26'd512 << FRAC_BITS
;

always_ff @(negedge clk or negedge reset_n) begin
    if (!reset_n) begin
        acc <= START_VAL;
    end else begin

        if (mppt_enable == 1'b0) begin
            // === MANUAL MODE ===
            acc <= signed'({1'b0, manual_duty_in}) <<
                FRAC_BITS;
        end
        else begin

```

```

// === MPPT MODE ===
logic signed [26:0] next_acc;
next_acc = acc + duty_adjust;

if (next_acc > MAX_ACC)
    acc <= MAX_ACC;
else if (next_acc < MIN_ACC)
    acc <= MIN_ACC;
else
    acc <= next_acc[25:0];
end
end
end

// === OUTPUT ===
// Dynamic Slicing
assign dac_value = acc[FRAC_BITS + 9 : FRAC_BITS];

endmodule

```

5.1.6. Fuzzy Top Module

The fuzzy top module serves as the highest-level structural wrapper for the digital controller. It instantiates and routes the signals between all the previously discussed components, including the difference generators, the fuzzifier core, and the duty integrator. It should be emphasized that these underlying modules are designed to be highly customizable. In a standard commercial deployment, their parameters (such as membership function thresholds and rule weights) would be dynamically configurable via a register bank accessed over an SPI or I2C kind of interface. While a dedicated communication protocol is not implemented in this work, these values are passed as structural **parameter** definitions at instantiation, fully preserving the design’s modularity. By seamlessly connecting these optimized sub-modules, the fuzzy top module

forms the complete, hardware-efficient Maximum Power Point Tracking (MPPT) control logic, fully prepared for physical synthesis. The SystemVerilog implementation of this top-level module is provided in Code Snippet 5.9.

Code Snippet 5.9. Fuzzy Top Module SystemVerilog Implementation

```
module fuzzy_top #(
    // dV ( $\Delta V_{pv}$ ) Membership Function Thresholds
    parameter signed [10:0] dV_A_NEG = -5,
    parameter signed [10:0] dV_B_NEG = 0,
    parameter signed [10:0] dV_A_TRI = -4,
    parameter signed [10:0] dV_B_TRI = 0,
    parameter signed [10:0] dV_C_TRI = 4,
    parameter signed [10:0] dV_A_POS = 0,
    parameter signed [10:0] dV_B_POS = 5,

    // dI ( $\Delta I_{out}$ ) Membership Function Thresholds
    parameter signed [10:0] dI_A_NEG = -5,
    parameter signed [10:0] dI_B_NEG = 0,
    parameter signed [10:0] dI_A_TRI = -4,
    parameter signed [10:0] dI_B_TRI = 0,
    parameter signed [10:0] dI_C_TRI = 4,
    parameter signed [10:0] dI_A_POS = 0,
    parameter signed [10:0] dI_B_POS = 5,

    // Vpv Membership Function Thresholds
    parameter signed [10:0] Vpv_A_LOW = 340,
    parameter signed [10:0] Vpv_B_LOW = 480,
    parameter signed [10:0] Vpv_A_OPT = 460,
    parameter signed [10:0] Vpv_B_OPT = 512,
    parameter signed [10:0] Vpv_C_OPT = 533,
    parameter signed [10:0] Vpv_A_HIGH = 530,
    parameter signed [10:0] Vpv_B_HIGH = 680,
```

```

// Sugeno Output Constants (Defuzzifier)
parameter signed [7:0] C1 = -80, // Big Decrease
parameter signed [7:0] C2 = 80, // Big Increase
parameter signed [7:0] C3 = -15, // Decrease
parameter signed [7:0] C4 = 15, // Increase
parameter signed [7:0] C5 = 15, // Increase
parameter signed [7:0] C6 = -15, // Decrease
parameter signed [7:0] C7 = 0, // Hold
parameter signed [7:0] C8 = 0 // Hold
)
(
    input logic clk,
    input logic reset_n,
    input logic mppt_enable,
    input logic [9:0] manual_duty_adj,
    input logic [9:0] V_in, // ADC Voltage (10-bit, Vcm
        at 512: Vpv/12 - 2.5V)
    input logic [9:0] I_in, // ADC Current (10-bit, Vcm
        at 512: I/8 - 2.5V)
    output logic [9:0] dac_value // Final DAC Output to
        hardware
);

// --- Internal Signals ---
logic signed [10:0] diff_V; // Change in Voltage
logic signed [10:0] diff_I; // Change in Current
logic signed [10:0] duty_adjust; // Fuzzy Output (+/-
    15, +/- 1, etc.)

// --- 1. Differentiators (Calculate dV and dI) ---
diff_generator diff_V_inst (
    .clk(clk),
    .rst_n(reset_n),

```



```

        .in_value(V_in),
        .diff(diff_V)
    );

diff_generator diff_I_inst (
    .clk(clk),
    .rst_n(reset_n),
    .in_value(I_in),
    .diff(diff_I)
);

// --- 2. The Fuzzy Brain ---
fuzzy_core fuzzy_core_inst (
    .clk(clk),
    .reset_n(reset_n),

    // Dynamic Inputs (Changes)
    .diff_V(diff_V),
    .diff_I(diff_I),

    // Raw Voltage Input (for Low/Optimal/High detection)
    .V_pv({1'b0, V_in}),

    // dV Membership Function Parameters
    .dV_A_neg(dV_A_NEG), .dV_B_neg(dV_B_NEG),
    .dV_A_tri(dV_A_TRI), .dV_B_tri(dV_B_TRI), .dV_C_tri(
        dV_C_TRI),
    .dV_A_pos(dV_A_POS), .dV_B_pos(dV_B_POS),

    // dI Membership Function Parameters
    .dI_A_neg(dI_A_NEG), .dI_B_neg(dI_B_NEG),
    .dI_A_tri(dI_A_TRI), .dI_B_tri(dI_B_TRI), .dI_C_tri(
        dI_C_TRI),

```

```

        .dI_A_pos(dI_A_POS), .dI_B_pos(dI_B_POS),

// Vpv Membership Function Parameters
        .Vpv_A_low(Vpv_A_LOW), .Vpv_B_low(Vpv_B_LOW),
        .Vpv_A_opt(Vpv_A_OPT), .Vpv_B_opt(Vpv_B_OPT), .
            Vpv_C_opt(Vpv_C_OPT),
        .Vpv_A_high(Vpv_A_HIGH), .Vpv_B_high(Vpv_B_HIGH),

// Sugeno Constants
        .C1(C1), .C2(C2), .C3(C3), .C4(C4),
        .C5(C5), .C6(C6), .C7(C7), .C8(C8),

// Output
        .duty_adjust(duty_adjust)
    );

// Inside your Top Module
logic [9:0] mppt_duty_wire; // Output of the Integrator

// 1. The Integrator
duty_integrator integrator_inst (
    .clk(clk),
    .reset_n(reset_n),
    .mppt_enable(mppt_enable), // The physical
        switch state
    .manual_duty_in(manual_duty_adj), // MANUAL INPUT
    .duty_adjust(duty_adjust),
    .dac_value(mppt_duty_wire)
);

// 2. The Switch (Multiplexer)
assign dac_value = (mppt_enable) ? mppt_duty_wire :
    manual_duty_adj;

```

```
endmodule
```

5.2. Parameter Configuration and SPI Integration

In a fully integrated mixed-signal System-on-Chip (SoC), a digital communication interface, such as a Serial Peripheral Interface (SPI), is typically employed to configure digital parameters and calibrate analog blocks. Such an interface would enable the dynamic adjustment of the fuzzy logic parameters—such as membership function thresholds and Sugeno output weights—as well as the calibration trims for the ADC, DAC, and ramp generator arrays.

However, the development of a complete SPI module and its associated register map falls outside the primary scope of this thesis, which focuses on the architectural optimization of the MPPT fuzzy logic controller. Therefore, a dedicated communication interface was not implemented. Instead, the fuzzy logic settings are hardcoded as structural `parameter` definitions at the top level of the `fuzzy_top` module. Similarly, the necessary trim and calibration signals for the analog front-end are supplied using ideal static sources within the Cadence simulation environment. This approach simplifies the verification testbench and design complexity while fully preserving the functional validation of the core MPPT algorithm and hardware-efficient architecture.

6. CONCLUSION

This thesis has presented the comprehensive modeling, design, and implementation of a Fuzzy Logic-based Maximum Power Point Tracking (MPPT) controller for photovoltaic battery charging systems. The primary objective was to develop an intelligent, highly responsive, and hardware-efficient MPPT solution capable of overcoming the limitations of conventional algorithms under dynamic environmental conditions.

The research began with high-level system modeling in MATLAB/Simulink, where a Sugeno-type fuzzy inference system was designed and validated. By evaluating three key parameters—change in PV voltage, change in output current, and absolute PV voltage—the controller demonstrated exceptional tracking accuracy and transient response. To transition this theoretical model towards a physical Integrated Circuit (IC), a mixed-signal architecture was developed within the Cadence Virtuoso environment. The analog front-end, encompassing a 12-bit asynchronous SAR ADC, an R-2R DAC, a high-speed continuous comparator, a ramp generator, and an LDO regulator, was designed to provide robust and precise interfacing between the continuous-time physical world and the discrete-time digital controller. Concurrently, the fuzzy logic algorithm was structurally optimized for digital synthesis. By replacing resource-intensive multipliers and dividers with bitwise shift operations and lookup tables, the digital footprint was drastically minimized without compromising the MPPT performance.

To validate the complete system architecture, an Analog Mixed-Signal (AMS) testbench was constructed. The fully integrated `IC_core`, which houses both the custom analog blocks and the synthesizable SystemVerilog digital codes, is illustrated in Figure 6.1. This core was then instantiated within a comprehensive top-level testbench (Figure 6.2), incorporating the Verilog-A photovoltaic panel model and the external buck converter power stage.

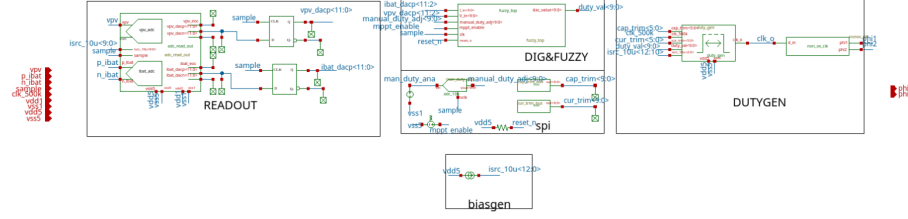


Figure 6.1. Top-level schematic of the fully integrated mixed-signal IC_core.

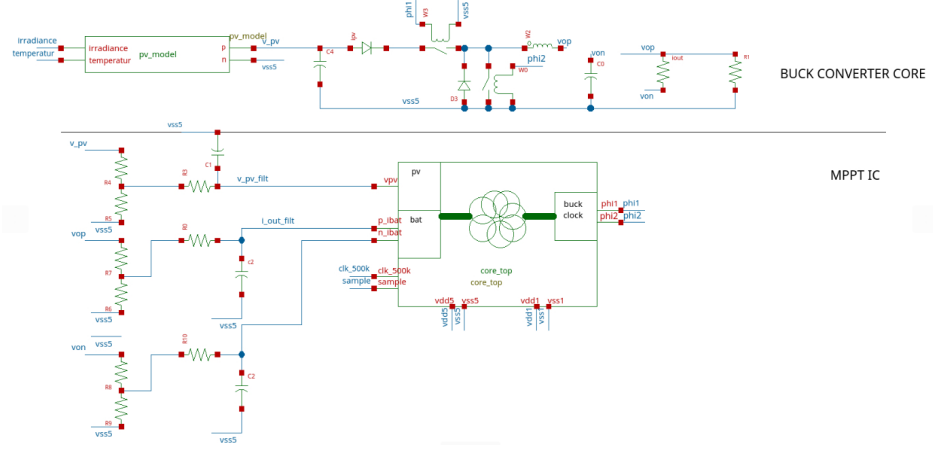


Figure 6.2. Complete AMS testbench including the IC_core, Verilog-A PV model, and buck converter.

The final AMS simulation results, presented in Figure 6.3, confirm the successful integration and functionality of the designed components. Upon activation, the digital fuzzy controller seamlessly processes the digitized inputs from the SAR ADC and modulates the duty cycle via the DAC and ramp generator. As demonstrated by the results, the PV voltage (V_{pv}) successfully converges to approximately 30.4 V, closely aligning with the theoretical V_{mpp} specified previously in the PV model parameters (Table 3.1).

Despite the complex mixed-signal interactions, the system maintains stable operation and high power conversion efficiency. A summary of the key performance metrics extracted from this final simulation is provided in Table 6.1. The controller achieves a rapid settling time of approximately 15 ms to reach the MPP, extracting 231 W of power from the PV panel. This translates to an exceptional PV tracking efficiency

of over 99%. Concurrently, the system delivers 222.3 W of output power to the load, demonstrating an excellent buck converter efficiency of 96.2%.

Table 6.1. Summary of Final AMS Simulation Performance

Parameter	Value
PV Operating Voltage (V_{pv})	~ 30.4 V
Settling Time to MPP	~ 15 ms
Extracted PV Power (P_{in})	231 W
PV Tracking Efficiency	$> 99\%$
Output Power (P_{out})	222.3 W
Buck Converter Efficiency	96.2%

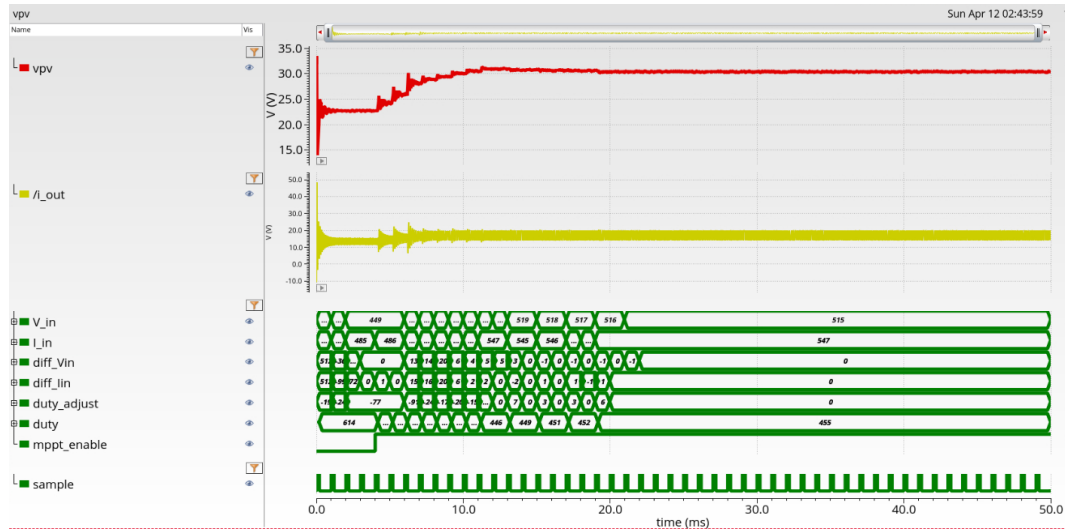


Figure 6.3. Final AMS simulation results demonstrating successful MPPT tracking.

In conclusion, this thesis successfully bridges the gap between advanced intelligent control algorithms and practical, low-cost silicon implementation. The proposed hardware-efficient fuzzy logic architecture, coupled with the robust analog front-end design, provides a highly effective solution for next-generation, fully integrated PV power management ICs. Future work will focus on the physical layout, tape-out, and subsequent experimental hardware validation of the fabricated prototype.

To promote reproducibility and encourage further research, the complete MAT-

LAB/Simulink models, SystemVerilog implementations, and associated code developed for this study are publicly available at <https://github.com/mehmetgok1/MATLAB>.

REFERENCES

1. Erickson, R. and D. Maksimovic, *Fundamentals of Power Electronics*, Springer, 2nd edn., 2001.
2. Rashid, M. H., *Power Electronics Handbook*, Butterworth-Heinemann, 4th edn., 2017.
3. Instruments, T., *Basic Calculation of a Buck Converter*, 2015, application Report SLVA477B.
4. Soto, W. D., S. Klein and W. Beckman, “Improvement and validation of a model for photovoltaic array performance”, *Solar Energy*, Vol. 80, No. 1, pp. 78–88, 2006.
5. Razavi, B., “The Bootstrapped Switch”, *IEEE Solid-State Circuits Magazine*, Vol. 7, No. 3, pp. 12–15, 2015.
6. Abo, A. M. and P. R. Gray, “A 1.5-V, 10-bit, 14.3-MS/s CMOS pipeline analog-to-digital converter”, *IEEE Journal of Solid-State Circuits*, Vol. 34, No. 5, pp. 599–606, 1999.
7. Fan, H., P. Lei, Q. Feng and Q. Wei, “Optimized Split Capacitive Array in 16-Bit SAR ADC with Redundancy”, *Circuits, Systems, and Signal Processing*, Vol. 42, No. 3, pp. 1264–1278, 2023.
8. Razavi, B., “The StrongARM Latch”, *IEEE Solid-State Circuits Magazine*, Vol. 5, No. 2, pp. 12–17, 2013.
9. Systems, C. D., “Noise Analysis of a Dynamic Comparator”, Cadence Chalk Talk, 2016, https://www.cadence.com/content/dam/cadence-www/global/en_US/videos/tools/custom-_ic_analog_rf_design/NoiseAnalyisposting201612Chalk%20Talk.pdf.

10. Luque, A. and S. Hegedus, *Handbook of Photovoltaic Science and Engineering*, John Wiley & Sons, 2nd edn., 2011.
11. Eswam, T. and P. L. Chapman, “Comparison of Photovoltaic Array Maximum Power Point Tracking Techniques”, *IEEE Transactions on Energy Conversion*, Vol. 22, No. 2, pp. 439–449, 2007.
12. Hohm, D. P. and M. E. Ropp, “Comparative study of maximum power point tracking algorithms”, *Progress in Photovoltaics: Research and Applications*, Vol. 11, No. 1, pp. 47–62, 2003.
13. Dalala, Z., Z. U. Zahid, H. Y. Kanaan, N. M. L. Ha and D. C. Habineza, “Design and real-time implementation of a novel photovoltaic MPPT system”, *IEEE Transactions on Sustainable Energy*, Vol. 7, No. 3, pp. 1101–1115, 2016.
14. Ubale, T. S. and S. V. Chaudhari, “Review on fuzzy logic based MPPT for photovoltaic systems”, *Proc. IEEE Int. Conf. Elect. Eng. Technol. (ICETECH)*, pp. 1–6, 2016.
15. MathWorks, *Simulink User’s Guide*, Natick, MA, USA, 2024.
16. Robles, C., J. Taborda and O. Rodríguez, “Fuzzy Logic Based MPPT Controller for a PV System”, *Energies*, Vol. 10, No. 12, p. 2036, 2017.
17. Iqbal, A. *et al.*, “Modeling and simulation of a photovoltaic system with MPPT”, *MSU Journal of Engineering*, 2016.
18. Sathya, P. and R. Natarajan, “Modelling and simulation of maximum power point tracking for PV system”, *IEEE International Conference on Clean Energy and Technologies*, 2013.